

Advanced Java

Interview Preparation — Deep Dive

9 Topics · Concepts · Code · Q&A

For Java backend developers (1-3 years experience)
Spring · JPA · Testing · Patterns · SOLID · JVM

Table of Contents

1. JDBC & Database Access
2. Spring Core: IoC & Dependency Injection
3. Spring Boot & REST APIs
4. Spring Data JPA & Hibernate
5. Spring Transactions
6. Testing — JUnit & Mockito
7. Design Patterns
8. SOLID Principles
9. JVM Internals

Topic 1 — JDBC & Database Access

1.1 What is JDBC?

JDBC (Java Database Connectivity) is a Java API that defines how clients interact with databases. It's a specification — actual implementations are provided by **JDBC drivers** (MySQL Connector/J, PostgreSQL JDBC, Oracle JDBC, etc.).

Architecture

Java App → JDBC API (java.sql) → JDBC Driver → Database

JDBC Driver Types

Type	Description	Example
Type 1	JDBC-ODBC bridge	Deprecated
Type 2	Native API (partly Java, uses native libs)	Older Oracle OCI driver
Type 3	Network protocol (Java to middleware)	Rare
Type 4	Pure Java (talks directly to DB)	MySQL Connector/J, PostgreSQL JDBC — most common

1.2 Fetching Data Without JPA (Pure JDBC)

The **6 standard steps** every interviewer wants to hear:

1. Load the driver (optional in JDBC 4.0+)
2. Establish connection (`DriverManager.getConnection`)
3. Create statement (`Statement` , `PreparedStatement` , or `CallableStatement`)
4. Execute query (`executeQuery` / `executeUpdate` / `execute`)
5. Process result (`ResultSet`)
6. Close resources (use try-with-resources!)

Full JDBC fetch example

```
import java.sql.*;
import java.util.*;

public class JdbcDemo {
    private static final String URL = "jdbc:mysql://localhost:3306/mydb";
    private static final String USER = "root";
    private static final String PASS = "password";

    public List<Employee> findAll() throws SQLException {
        List<Employee> employees = new ArrayList<>();
        String sql = "SELECT id, name, dept FROM employees";

        try (Connection conn = DriverManager.getConnection(URL, USER, PASS);
            PreparedStatement ps = conn.prepareStatement(sql);
            ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {
                Employee e = new Employee();
                e.setId(rs.getInt("id"));
                e.setName(rs.getString("name"));
                e.setDept(rs.getString("dept"));
                employees.add(e);
            }
        }
        return employees;
    }
}
```

Connection given URL, user, password

```
// Pre-Java 6 style – load driver explicitly
Class.forName("com.mysql.cj.jdbc.Driver");

// Modern (Java 6+) – auto-loaded
Connection conn = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/mydb", "root", "password");
```

URL formats

```
jdbc:mysql://host:3306/dbname
jdbc:postgresql://host:5432/dbname
jdbc:oracle:thin:@host:1521:SID
jdbc:sqlserver://host:1433;databaseName=dbname
jdbc:h2:mem:testdb
```

1.3 Statement vs PreparedStatement vs CallableStatement

Feature	Statement	PreparedStatement	CallableStatement
SQL	Static, hardcoded	Parameterized (? placeholders)	For stored procedures
Pre-compiled	✗ Re-parsed each time	✓ Compiled once, reused	✓
SQL Injection safe	✗ No	✓ Yes	✓
Performance	Slower	Faster	Same
Use case	Rare — DDL like CREATE TABLE	Default for queries	Stored procedure calls

1.4 Why PreparedStatement?

Advantages

1. **Prevents SQL Injection** (security)
2. **Performance** — pre-compiled, reusable
3. **Cleaner code** — no manual escaping
4. **Type-safe parameter binding** (setInt, setString, setDate, setNull)
5. **Handles binary/large data** (setBlob, setClob, setBytes)
6. **Better readable SQL** — separates query from data

1.5 SQL Injection — The SECURITY Answer

SQL Injection is when malicious user input is concatenated into SQL, allowing attackers to alter the query's intent.

✗ Vulnerable code with Statement

```
String username = request.getParameter("username");
String sql = "SELECT * FROM users WHERE username = '" + username + "'";
Statement st = conn.createStatement();
ResultSet rs = st.executeQuery(sql);
```

If user enters `' OR '1'='1`:

```
SELECT * FROM users WHERE username = '' OR '1'='1'
```

→ Returns ALL users. Auth bypassed.

Worse — `'; DROP TABLE users; --` drops the table. ☹

✓ Safe code with PreparedStatement

```
String sql = "SELECT * FROM users WHERE username = ?";
PreparedStatement ps = conn.prepareStatement(sql);
ps.setString(1, username); // value bound, treated as DATA not SQL
ResultSet rs = ps.executeQuery();
```

Even if input is `' OR '1'='1`, the DB driver treats it as a literal string value — not part of SQL syntax. **Injection impossible.**

How PreparedStatement prevents injection: The query is parsed and compiled BEFORE parameters are supplied. The DB knows the structure of the query at parse time. Parameters are passed separately as typed values, never re-parsed as SQL.

1.6 PreparedStatement Parameter Binding

```
String sql = "INSERT INTO employees (id, name, dept, salary, joined_on) VALUES (?, ?, ?, ?, ?)";
try (PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setInt(1, 101);
    ps.setString(2, "Raj");
    ps.setString(3, "CSE");
    ps.setDouble(4, 50000.0);
    ps.setDate(5, java.sql.Date.valueOf(LocalDate.now()));
    int rows = ps.executeUpdate();
}
}
```

Common setter methods

Method	Use for
setInt, setLong, setDouble, setFloat	Numbers
setString	Text
setBoolean	Boolean
setDate, setTime, setTimestamp	Date/time
setBigDecimal	Precise decimals (money!)
setBytes, setBlob	Binary data
setNull(idx, Types.INTEGER)	NULL with type hint
setObject(idx, value)	Auto-detect type

1.7 Execute Methods

Method	Returns	Use for
executeQuery()	ResultSet	SELECT
executeUpdate()	int (rows affected)	INSERT, UPDATE, DELETE, DDL
execute()	boolean (true if ResultSet)	Any SQL — when type unknown
executeBatch()	int[]	Batch operations

1.8 Batch Operations

```
String sql = "INSERT INTO employees (id, name) VALUES (?, ?)";
try (PreparedStatement ps = conn.prepareStatement(sql)) {
    conn.setAutoCommit(false);
    for (Employee e : list) {
        ps.setInt(1, e.getId());
        ps.setString(2, e.getName());
        ps.addBatch();
        if (i % 1000 == 0) ps.executeBatch();
    }
    ps.executeBatch();
    conn.commit();
}
}
```

Reduces network round-trips dramatically.

1.9 Transactions in JDBC

By default, JDBC is in **auto-commit mode** — every statement is its own transaction.

```
try (Connection conn = DriverManager.getConnection(URL, USER, PASS)) {
    conn.setAutoCommit(false); // start transaction
    try {
        debit(conn, fromAccount, amount);
        credit(conn, toAccount, amount);
        conn.commit(); // success — make permanent
    } catch (SQLException e) {
        conn.rollback(); // failure — undo all
        throw e;
    }
}
}
```

Isolation Levels

```
conn.setTransactionIsolation(Connection.TRANSACTION_READ_COMMITTED);
```

Level	Dirty Read	Non-Repeatable Read	Phantom Read
READ_UNCOMMITTED	Possible	Possible	Possible
READ_COMMITTED	✗	Possible	Possible
REPEATABLE_READ	✗	✗	Possible
SERIALIZABLE	✗	✗	✗

1.10 ResultSet — Navigation & Access

```
while (rs.next()) {                                // forward iteration
    int id = rs.getInt("id");                        // by column name
    String name = rs.getString(2);                  // by 1-based index
    boolean wasNull = rs.wasNull();
}

// Scrollable ResultSet
PreparedStatement ps = conn.prepareStatement(sql,
    ResultSet.TYPE_SCROLL_INSENSITIVE,
    ResultSet.CONCUR_READ_ONLY);
rs.first(); rs.last(); rs.previous(); rs.absolute(5);
```

ResultSetMetaData

```
ResultSetMetaData meta = rs.getMetaData();
int cols = meta.getColumnCount();
String name = meta.getColumnName(1);
String type = meta.getColumnTypeName(1);
```

1.11 Connection Pooling (Critical for production!)

Creating a JDBC connection is **expensive** (TCP handshake, auth). Connection pools maintain a cache of open connections.

Pool	Notes
HikariCP	Default in Spring Boot. Fastest, most popular.
Apache DBCP	Older, slower
C3P0	Older
Tomcat JDBC Pool	Built into Tomcat

HikariCP example

```
HikariConfig config = new HikariConfig();
config.setJdbcUrl("jdbc:mysql://localhost:3306/mydb");
config.setUsername("root");
config.setPassword("password");
config.setMaximumPoolSize(10);
HikariDataSource ds = new HikariDataSource(config);

try (Connection conn = ds.getConnection()) {
    // use connection – returned to pool on close
}
```

In Spring Boot

```
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=password
spring.datasource.hikari.maximum-pool-size=20
```

HikariCP is auto-configured.

1.12 try-with-resources (essential!)

```
try (Connection conn = DriverManager.getConnection(...);
    PreparedStatement ps = conn.prepareStatement(sql);
    ResultSet rs = ps.executeQuery()) {
    // auto-closed in REVERSE order
}
```

1.13 SQLException

Checked exception. Always wrapped or handled.

```
try { /* jdbc code */ }
catch (SQLException e) {
    log.error("SQL error: {}", e.getMessage());
    log.error("SQL State: {}", e.getSQLState());      // standard 5-char code
    log.error("Error Code: {}", e.getErrorCode());   // vendor-specific
}
```

Spring wraps SQLException in `DataAccessException` (unchecked).

1.14 Top JDBC Q&A

Q: What is JDBC?

Java API for interacting with relational databases. A specification implemented by vendor-specific drivers.

Q: Steps to connect to a DB without JPA?

1) Load driver (auto since JDBC 4.0). 2) Get Connection via DriverManager. 3) Create PreparedStatement. 4) Execute query → ResultSet. 5) Iterate ResultSet. 6) Close everything (try-with-resources).

Q: Why use PreparedStatement?

Prevents SQL injection (binds parameters as data); pre-compiled (better performance); cleaner code (no escaping); type-safe parameter setters; handles binary/large data well.

Q: What's the SECURITY benefit of PreparedStatement?

Prevents SQL Injection. With Statement, user input is concatenated into SQL → attackers can inject malicious code. PreparedStatement parses the query first, then binds parameters as typed data — DB never re-parses the input as SQL.

Q: Statement vs PreparedStatement?

Statement: static SQL, vulnerable to injection, slower, re-parsed each time. PreparedStatement: parameterized, safe, pre-compiled, faster.

Q: Difference between executeQuery, executeUpdate, execute?

executeQuery → ResultSet (SELECT). executeUpdate → int rows affected (INSERT/UPDATE/DELETE/DDDL). execute → boolean (general purpose).

Q: What is connection pooling? Why use it?

A pool of pre-opened connections reused across requests. Creating connections is expensive (TCP+auth). Pools (HikariCP) cut latency dramatically.

Q: Default JDBC isolation level?

Database-dependent. MySQL → REPEATABLE_READ. Postgres/Oracle → READ_COMMITTED.

Q: How to handle transactions in JDBC?

`conn.setAutoCommit(false)`, perform operations, `conn.commit()` on success or `conn.rollback()` on failure.

Q: SQLException — checked or unchecked?

Checked. Spring wraps it in unchecked `DataAccessException` for cleaner code.

Q: What happens if you don't close a Connection?

Resource leak — connection stays open, eventually pool/DB runs out → app fails. Always use try-with-resources.

Q: Difference between DriverManager and DataSource?

DriverManager: simple, creates new connection each time, no pooling. DataSource: enterprise — supports pooling, distributed transactions, configured externally.

Q: What is JDBC Template (Spring)?

Spring's wrapper that simplifies JDBC: handles boilerplate (open/close, exception conversion), provides `query` , `update` , `queryForObject` methods.

Topic 2 — Spring Core: IoC & Dependency Injection

2.1 What is Spring?

Spring Framework is a comprehensive Java framework for building enterprise applications. Its core promise: simplify Java development through IoC, DI, AOP, declarative transactions, templates.

Spring ecosystem overview

Module	Purpose
Spring Core	IoC container, DI
Spring MVC	Web framework
Spring Boot	Auto-configuration + opinionated defaults
Spring Data	Repository abstractions over JPA, MongoDB, Redis
Spring Security	Auth/authorization
Spring Cloud	Microservices toolkit
Spring Batch	Batch processing

2.2 Inversion of Control (IoC)

IoC = inverting the control of object creation and lifecycle from your code to a framework (the container).

Traditional approach (without IoC)

```
public class OrderService {
    private PaymentService paymentService = new PaymentService(); // tight coupling
    private EmailService emailService = new EmailService();        // hard to test/mock
}
```

IoC approach

```
public class OrderService {
    private final PaymentService paymentService;
    private final EmailService emailService;

    public OrderService(PaymentService p, EmailService e) {
        this.paymentService = p;
        this.emailService = e;
    }
}
```

Spring container creates dependencies and injects them. **You don't** **new** anymore.

2.3 Dependency Injection (DI)

DI = a design pattern that implements IoC. Dependencies are "injected" into a class rather than created inside it.

Three types of DI

1. Constructor Injection (✓ Recommended)

```
@Service
public class OrderService {
    private final PaymentService paymentService;

    @Autowired // optional in Spring 4.3+ if only one constructor
    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

2. Setter Injection

```

@Service
public class OrderService {
    private PaymentService paymentService;

    @Autowired
    public void setPaymentService(PaymentService p) {
        this.paymentService = p;
    }
}

```

3. Field Injection (X Discouraged)

```

@Service
public class OrderService {
    @Autowired
    private PaymentService paymentService;
}

```

Comparison

Type	Pros	Cons
Constructor	Immutable, mandatory deps explicit, easy testing, fails fast on missing deps	Verbose for many deps
Setter	Optional dependencies, can re-inject	Mutable, can leave object half-initialized
Field	Concise	Hidden deps, hard to test, can't be final, requires reflection

Best practice: Use constructor injection. Industry standard since Spring 4.3+.

2.4 Why DI? Why not create objects manually?

Manual creation problems

1. Tight coupling — `new XService()` ties you to a specific implementation forever
2. Code duplication — every place that needs the dep must `new` it
3. Testing pain — can't mock; tests become integration tests
4. Lifecycle issues — when do you create? When do you destroy? Singleton? Per request?
5. Configuration scattered — DB connection, properties, etc. set up in 10 places
6. Memory waste — unnecessary multiple instances of stateless services
7. Wiring graph complexity — A needs B needs C needs D — manually wiring this gets hairy

With DI

- Spring instantiates beans **once** (singleton scope by default)
- Dependencies wired automatically based on type
- Mockable in tests via `@MockBean` or constructor injection of mocks
- Configuration centralized (annotations or XML or `@Configuration` classes)

Concrete example showing the win

```

// x Manual — tightly coupled, untestable
public class OrderService {
    private PaymentService payment = new StripePaymentService();
    private EmailService email = new SmtEmailService();
    private InventoryService inv = new InventoryService(
        new DatabaseConnection("jdbc:...")); // dependency chain!
}

// ✓ DI — loose, testable, configurable
@Service
public class OrderService {
    private final PaymentService payment;
    private final EmailService email;
    private final InventoryService inv;

    public OrderService(PaymentService p, EmailService e, InventoryService i) {
        this.payment = p; this.email = e; this.inv = i;
    }
}

```

2.5 Spring IoC Container

Interface	Purpose
BeanFactory	Basic container — lazy initialization
ApplicationContext	Extends BeanFactory + enterprise features (events, AOP, i18n, environment)

99% of the time, use ApplicationContext.

Common ApplicationContext implementations

- AnnotationConfigApplicationContext — Java config
- ClassPathXmlApplicationContext — XML config
- WebApplicationContext — for web apps
- AnnotationConfigWebApplicationContext — Java config + web

2.6 Beans

A Bean is an object managed by the Spring IoC container. Spring creates it, configures it, manages its lifecycle, and injects it where needed.

Method 1: Stereotype annotations (auto-detected via component scan)

```
@Component // generic
@Service // service layer
@Repository // data layer (also auto-translates SQL exceptions)
@Controller // web controller
@RestController // @Controller + @ResponseBody
@Configuration // declares config + bean methods
```

All are @Component underneath — different names for clarity and AOP targeting.

Method 2: @Bean in @Configuration class

```
@Configuration
public class AppConfig {

    @Bean
    public PaymentService paymentService() {
        return new StripePaymentService();
    }

    @Bean
    public OrderService orderService(PaymentService paymentService) {
        return new OrderService(paymentService);
    }
}
```

When to use @Bean over stereotypes: third-party classes (you can't add @Component), multiple beans of same type with different config, conditional creation, external library wiring.

2.7 Component Scanning

```
@Configuration
@ComponentScan(basePackages = "com.myapp")
public class AppConfig { }
```

Spring Boot: `@SpringBootApplication` already includes `@ComponentScan` for the package it's in (and sub-packages).

2.8 @Autowired and Resolution

By default, **by type**. If multiple beans of same type exist → ambiguity error.

Disambiguating with @Qualifier

```

@Service("stripe")
public class StripePaymentService implements PaymentService { }

@Service("paypal")
public class PaypalPaymentService implements PaymentService { }

@Service
public class OrderService {
    public OrderService(@Qualifier("stripe") PaymentService payment) { ... }
}

```

@Primary — designate a default

```

@Service
@Primary
public class StripePaymentService implements PaymentService { }

```

Other annotations

Annotation	Purpose
@Autowired	Spring's DI annotation
@Inject	JSR-330 standard equivalent
@Resource	JSR-250, by name first
@Qualifier	Disambiguate by name
@Primary	Mark default bean
@Value("\${prop}")	Inject property values

2.9 Bean Scopes

Scope	Description
singleton (default)	One instance per Spring container — shared
prototype	New instance every time it's requested
request	One per HTTP request (web only)
session	One per HTTP session (web only)
application	One per ServletContext (web only)
websocket	One per WebSocket session

```

@Service
@Scope("prototype")
public class ReportGenerator { }

```

Singleton trap: Singleton beans must be stateless (no mutable instance fields shared across threads). Otherwise → race conditions.

Singleton injecting Prototype — common pitfall: The prototype is created once (when the singleton is created), then re-used. Fix: inject `ObjectFactory<T>` or `Provider<T>` and call `getObject()`.

2.10 Bean Lifecycle

1. Instantiation (constructor)
2. Populate properties (DI happens)
3. `setBeanName()`, `setBeanFactory()`, `setApplicationContext()` — Aware callbacks
4. `@PostConstruct` method called
5. BEAN IS READY
... bean lives ...
6. `@PreDestroy` — on container shutdown

```
@Component
public class Cache {
    @PostConstruct
    public void load() { /* warm up cache */ }

    @PreDestroy
    public void cleanup() { /* save state, release resources */ }
}
```

2.11 @Configuration vs @Component

```
@Configuration
public class AppConfig {
    @Bean public B b() { return new B(); }
    @Bean public C c() { return new C(b()); } // calls b() – what happens?
}
```

In **@Configuration**: Spring creates a CGLIB **proxy** of the class. Calls to **b()** from within **c()** are intercepted to return the **same singleton** bean — not a new instance.

In **@Component**: No proxy. **b()** would return a **new instance** every time.

→ **Always use @Configuration for @Bean definitions**, not @Component.

2.12 Property Injection — @Value

```
# application.properties
app.name=MyApp
app.maxRetries=3
```

```
@Service
public class MyService {
    @Value("${app.name}")
    private String appName;

    @Value("${app.maxRetries:5}") // default 5 if missing
    private int maxRetries;
}
```

For type-safe configuration, prefer **@ConfigurationProperties**:

```
@Component
@ConfigurationProperties(prefix = "app")
public class AppProperties {
    private String name;
    private int maxRetries;
    // getters/setters
}
```

2.13 Profile-based Configuration

```
@Service
@Profile("dev")
public class FakePaymentService implements PaymentService { }

@Service
@Profile("prod")
public class StripePaymentService implements PaymentService { }
```

Activate via: **--spring.profiles.active=dev**

2.14 Top Spring Core Q&A

Q: What is Inversion of Control (IoC)?

Design principle where the framework controls object creation and lifecycle, instead of your code. Frees your code from "wiring" responsibilities.

Q: What is Dependency Injection (DI)?

Pattern that implements IoC — dependencies are provided ("injected") to a class instead of being created by it. Achieves loose coupling and testability.

Q: Difference between IoC and DI?

IoC is the principle; DI is the implementation pattern. IoC is broader — Service Locator is another IoC pattern, but Spring uses DI.

Q: Types of DI?

Constructor (preferred), setter, field. Constructor enables immutable, mandatory deps; setter for optional; field is concise but bad for testing.

Q: Why prefer constructor injection?

Immutability (final fields); mandatory deps explicit; fails fast at startup if missing; no reflection needed; easy to instantiate in tests without Spring.

Q: Why is field injection bad?

Hidden dependencies; can't make fields final; hard to test without Spring; encourages too many dependencies.

Q: What is a Spring Bean?

An object instantiated, configured, and managed by the Spring IoC container.

Q: BeanFactory vs ApplicationContext?

ApplicationContext extends BeanFactory + enterprise features: AOP, events, i18n, environment, eager init by default. Always use ApplicationContext.

Q: Default bean scope?

Singleton — one instance per Spring container.

Q: Singleton vs Prototype?

Singleton: one instance shared. Prototype: new instance per request from container. Singleton must be stateless.

Q: Difference between @Component, @Service, @Repository, @Controller?

All are @Component variants. Different names for clarity. @Repository adds exception translation. @Controller adds web semantics. @Service is for business logic.

Q: @Autowired — by type or by name?

By type by default. If multiple beans match → error. Disambiguate with @Qualifier("name") or mark one @Primary.

Q: How does Spring resolve circular dependencies?

For singleton scope, simple field/setter circular deps via early bean references (3-level cache). Constructor injection circular deps will fail. Solution: redesign or use @Lazy.

Q: @Configuration vs @Component for @Bean methods?

@Configuration proxies the class so @Bean method calls return the same singleton. With @Component, calling a @Bean method directly returns a new instance.

Q: Bean lifecycle stages?

Instantiate → populate properties → Aware callbacks → @PostConstruct → ready → @PreDestroy on shutdown.

Q: What is component scanning?

Spring scans specified packages for stereotype-annotated classes and registers them as beans. @ComponentScan configures it. Spring Boot includes it via @SpringBootApplication.

Q: Singleton injecting Prototype — what's the issue?

The prototype is injected once (at singleton creation), then reused — defeating prototype scope. Fix: inject ObjectFactory or Provider and call getObject() per use.

Q: How does Spring handle thread safety for singleton beans?

It doesn't. Singleton beans are inherently shared across threads. You must keep them stateless. This is why services typically have only final injected dependencies and no state.

Q: What is BeanPostProcessor?

Hook to intercept bean initialization. Called for every bean. Spring's AOP, @Transactional, validation all use this mechanism.

Q: What if no bean of required type is found?

NoSuchBeanDefinitionException at startup (fail-fast). Use `@Autowired(required = false)` for optional deps, or wrap in `Optional<T>`.

Topic 3 — Spring Boot & REST APIs

3.1 What is Spring Boot?

Spring Boot is an opinionated framework on top of Spring that eliminates boilerplate. It does three things:

- 1. **Auto-configuration** — sensible defaults based on classpath
- 2. **Starter dependencies** — curated dependency bundles
- 3. **Embedded server** — Tomcat/Jetty/Undertow built in

Spring vs Spring Boot

Spring	Spring Boot
Manual configuration (XML or Java)	Auto-configuration
External server (deploy WAR to Tomcat)	Embedded server (executable JAR)
Choose & manage dependency versions	Curated starters
More flexible, more setup	Opinionated, faster setup

Anatomy of a Spring Boot app

```
@SpringBootApplication
public class MyApp {
    public static void main(String[] args) {
        SpringApplication.run(MyApp.class, args);
    }
}
```

@SpringBootApplication = @Configuration + @EnableAutoConfiguration + @ComponentScan

3.2 REST API Basics

REST (Representational State Transfer) — architectural style for web APIs, based on HTTP.

REST Principles

- 1. **Stateless** — server holds no client state between requests
- 2. **Resource-based** — URIs identify resources (/users/123)
- 3. **Standard HTTP methods** — GET, POST, PUT, PATCH, DELETE
- 4. **Standard status codes** — 200, 201, 400, 404, 500
- 5. **Representation** — typically JSON
- 6. **HATEOAS** (optional) — responses include navigation links

HTTP Methods & Idempotency

Method	Purpose	Idempotent?	Safe?
GET	Read resource	✓	✓
POST	Create new resource	✗	✗
PUT	Replace resource (full update)	✓	✗
PATCH	Partial update	✗ (debated)	✗
DELETE	Remove resource	✓	✗

Resource URI design (best practices)

Bad	Good
/getAllUsers	GET /users
/getUserById?id=1	GET /users/1
/createUser	POST /users
/updateUser	PUT /users/1
/deleteUser?id=1	DELETE /users/1
/userOrders?uid=1	GET /users/1/orders

Rules: plural nouns, no verbs, hierarchy via path, query params for filtering/sorting/pagination.

3.3 Common HTTP Status Codes

Range	Meaning	Common codes
2xx	Success	200 OK, 201 Created, 204 No Content, 202 Accepted
3xx	Redirection	301 Moved Permanently, 304 Not Modified
4xx	Client error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 405 Method Not Allowed, 409 Conflict, 422 Unprocessable Entity, 429 Too Many Requests
5xx	Server error	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable, 504 Gateway Timeout

When to use which

- **200 OK** — successful GET, PUT, PATCH, DELETE
- **201 Created** — successful POST that creates resource (return Location header)
- **204 No Content** — successful DELETE (no body)
- **400 Bad Request** — invalid input/payload
- **401 Unauthorized** — not authenticated
- **403 Forbidden** — authenticated but not allowed
- **404 Not Found** — resource doesn't exist
- **409 Conflict** — duplicate (e.g., username taken)
- **500 Internal Server Error** — unexpected server crash

3.4 Spring MVC Controllers

```
@Controller    // returns view name (for template engines)
@ResponseBody  // serialize return as response body (JSON)

// Combined as:
@RestController = @Controller + @ResponseBody
```

For REST APIs, **always use @RestController**.

3.5 Path Mapping Annotations

Annotation	Use for	HTTP Method
@GetMapping	Read	GET
@PostMapping	Create	POST
@PutMapping	Update (full replace)	PUT
@PatchMapping	Update (partial)	PATCH
@DeleteMapping	Remove	DELETE
@RequestMapping	Generic — specify method	Any

The shortcuts (@GetMapping etc.) were introduced in Spring 4.3 for cleaner code.

Full CRUD example — Update Person Details

```

@RestController
@RequestMapping("/api/persons")
public class PersonController {

    private final PersonService personService;
    public PersonController(PersonService p) { this.personService = p; }

    // CREATE
    @PostMapping
    public ResponseEntity<Person> create(@RequestBody @Valid Person person) {
        Person saved = personService.save(person);
        return ResponseEntity.status(HttpStatus.CREATED).body(saved);
    }

    // READ ALL
    @GetMapping
    public List<Person> getAll() { return personService.findAll(); }

    // READ BY ID
    @GetMapping("/{id}")
    public ResponseEntity<Person> getById(@PathVariable Long id) {
        return personService.findById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    // UPDATE (full replace)
    @PutMapping("/{id}")
    public ResponseEntity<Person> update(@PathVariable Long id,
                                         @RequestBody @Valid Person person) {
        return ResponseEntity.ok(personService.update(id, person));
    }

    // PATCH (partial update)
    @PatchMapping("/{id}")
    public ResponseEntity<Person> partialUpdate(@PathVariable Long id,
                                                @RequestBody Map<String, Object> updates) {
        return ResponseEntity.ok(personService.partialUpdate(id, updates));
    }

    // DELETE
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        personService.delete(id);
        return ResponseEntity.noContent().build();
    }
}

```

3.6 Request Parameter Annotations

@PathVariable — for path segments

```

@GetMapping("/users/{id}")
public User getUser(@PathVariable Long id) { ... }

@GetMapping("/users/{userId}/orders/{orderId}")
public Order getOrder(@PathVariable Long userId, @PathVariable Long orderId) { ... }

```

@RequestParam — for query parameters (?key=value)

```

// /search?keyword=java&page=0&size=10
@GetMapping("/search")
public List<User> search(
    @RequestParam String keyword,
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "10") int size,
    @RequestParam(required = false) String sortBy) { ... }

```

@RequestBody — for JSON/XML body

```

@PostMapping("/users")
public User create(@RequestBody User user) { ... }

```

Spring uses **Jackson** to deserialize JSON → Java object.

@RequestHeader — for HTTP headers

```
@GetMapping("/profile")
public User profile(@RequestHeader("Authorization") String authHeader) { ... }
```

@RequestParam vs **@PathVariable**: **PathVariable** identifies a resource (`/users/{id}`); **RequestParam** filters/options.

3.7 ResponseEntity — Full Control Over HTTP Response

```
return ResponseEntity.ok(user); // 200 + body
return ResponseEntity.status(HttpStatus.CREATED).body(user); // 201 + body
return ResponseEntity.notFound().build(); // 404, no body
return ResponseEntity.noContent().build(); // 204, no body
return ResponseEntity.badRequest().body("Invalid input"); // 400 + body

// With custom headers
return ResponseEntity.ok()
    .header("X-Custom-Header", "value")
    .body(user);

// CREATED with Location header (REST best practice for POST)
return ResponseEntity.created(URI.create("/users/" + saved.getId())).body(saved);
```

3.8 Validation with @Valid

Spring integrates with **Bean Validation API** (JSR-380) — annotations on DTO fields enforce rules.

```
public class UserDTO {
    @NotNull
    @Size(min = 2, max = 50)
    private String name;

    @NotBlank
    @Email
    private String email;

    @NotNull
    @Min(18) @Max(120)
    private int age;

    @Pattern(regexp = "\\d{10}")
    private String phone;

    @Past
    private LocalDate dob;

    @Positive
    private double salary;

    @NotEmpty
    private List<String> roles;
}
```

Triggering validation in controller — @Valid

```
@PostMapping("/users")
public ResponseEntity<User> create(@RequestBody @Valid UserDTO dto) { ... }
```

If validation fails → `MethodArgumentNotValidException` thrown.

3.9 Global Exception Handling — @ControllerAdvice

```

@RestControllerAdvice // = @ControllerAdvice + @ResponseBody
public class GlobalExceptionHandler {

    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(UserNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ErrorResponse("USER_NOT_FOUND", ex.getMessage()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>> handleValidation(MethodArgumentNotValidException ex) {
        Map<String, String> errors = new HashMap<>();
        ex.getBindingResult().getFieldErrors()
            .forEach(e -> errors.put(e.getField(), e.getDefaultMessage()));
        return ResponseEntity.badRequest().body(errors);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleAll(Exception ex) {
        log.error("Unhandled exception", ex);
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(new ErrorResponse("INTERNAL_ERROR", "An unexpected error occurred"));
    }
}

```

@ResponseStatus — simpler alternative for custom exceptions

```

@ResponseStatus(HttpStatus.NOT_FOUND)
public class UserNotFoundException extends RuntimeException {
    public UserNotFoundException(String msg) { super(msg); }
}

```

3.10 REST API Layered Architecture

```

Controller (REST endpoints, validation)
    ↓
Service (business logic, transactions)
    ↓
Repository (data access)
    ↓
Database

```

3.11 DTO vs Entity (Best Practice)

Don't expose JPA entities directly via REST! Reasons:

- Entities have JPA proxies/lazy loading → JSON serialization fails or triggers extra queries
- Coupling DB schema with API contract
- Sensitive fields (password hashes) accidentally exposed
- Cyclic relationships cause infinite loops in JSON

```

// Entity (DB model)
@Entity
public class User {
    @Id @GeneratedValue private Long id;
    private String name;
    private String email;
    private String passwordHash; // ✗ should never reach client
    @OneToMany(mappedBy = "user") private List<Order> orders;
}

// DTO (API model)
public record UserDTO(Long id, String name, String email) { }

// Mapper
public class UserMapper {
    public static UserDTO toDto(User user) {
        return new UserDTO(user.getId(), user.getName(), user.getEmail());
    }
}

```

For complex mapping, use **MapStruct** or **ModelMapper**.

3.12 Pagination & Sorting

```
@GetMapping("/users")
public Page<User> getUsers(Pageable pageable) {
    return userRepo.findAll(pageable);
}
// Call: GET /users?page=0&size=10&sort=name,asc
```

3.13 Spring Boot Configuration

```
server.port=8080
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=secret
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

Profile-specific files

- `application-dev.properties`
- `application-prod.properties`

Activate via: `--spring.profiles.active=prod`

3.14 Spring Boot Actuator

Adds endpoints for health, metrics, env, etc. (default at `/actuator`):

- `/actuator/health` — health check
- `/actuator/info` — app info
- `/actuator/metrics` — performance metrics
- `/actuator/env` — environment props
- `/actuator/loggers` — change log levels at runtime

3.15 CORS Handling

```
// Method-level
@CrossOrigin(origins = "http://localhost:3000")
@GetMapping("/users")
public List<User> getAll() { ... }

// Class-level
@CrossOrigin(origins = "*")
@RestController
public class UserController { ... }

// Global config
@Configuration
public class CorsConfig {
    @Bean
    public WebMvcConfigurer corsConfigurer() {
        return new WebMvcConfigurer() {
            @Override
            public void addCorsMappings(CorsRegistry registry) {
                registry.addMapping("/api/**")
                    .allowedOrigins("http://localhost:3000")
                    .allowedMethods("GET", "POST", "PUT", "DELETE");
            }
        };
    }
}
```

3.16 HTTP Client (RestTemplate / WebClient)

RestTemplate (synchronous)

```

@Service
public class ExternalApiClient {
    private final RestTemplate restTemplate;

    public ExternalApiClient(RestTemplateBuilder builder) {
        this.restTemplate = builder.build();
    }

    public User fetchUser(Long id) {
        return restTemplate.getForObject(
            "https://api.example.com/users/{id}", User.class, id);
    }
}

```

WebClient (reactive, modern — Spring 6+)

```

WebClient client = WebClient.create("https://api.example.com");
User user = client.get()
    .uri("/users/{id}", id)
    .retrieve()
    .bodyToMono(User.class)
    .block();

```

3.17 Top REST API Q&A

Q: What is REST?

Representational State Transfer — architectural style for web services using HTTP methods, resource-based URIs, statelessness, and standard status codes.

Q: REST principles?

Statelessness, resource-based URIs, standard HTTP methods, standard status codes, representation (JSON/XML), HATEOAS (optional).

Q: What annotation maps a URL path to a method to UPDATE person details?

@PutMapping for full update, **@PatchMapping** for partial. With **@PathVariable** for the ID.

Q: Difference between PUT and PATCH?

PUT replaces the entire resource (full update). PATCH applies partial changes. PUT is idempotent; PATCH usually is too in practice.

Q: Difference between PUT and POST?

POST creates a new resource (server assigns ID). PUT replaces a resource at a known URI (client-known ID). POST is not idempotent; PUT is.

Q: @PathVariable vs @RequestParam?

PathVariable extracts from URL path. RequestParam extracts query parameters. PathVariable identifies a resource; RequestParam filters/options.

Q: @RequestBody — what does it do?

Tells Spring to deserialize the HTTP request body (JSON/XML) into a Java object using a `HttpMessageConverter` (Jackson by default).

Q: @RestController vs @Controller?

@RestController = **@Controller** + **@ResponseBody**. **RestController** returns serialized objects; **Controller** returns view names.

Q: Difference between @RequestMapping and @GetMapping?

@GetMapping is a Spring 4.3+ shortcut for **@RequestMapping(method = RequestMethod.GET)**. Cleaner.

Q: How does Spring serialize a Java object to JSON?

Via Jackson (default `HttpMessageConverter`). Add `spring-boot-starter-web` and Jackson is auto-configured.

Q: How to validate request body?

Annotate DTO fields (**@NotNull**, **@Size**, **@Email**), use **@Valid** on the controller parameter. Failures throw `MethodArgumentNotValidException` → handle in **@RestControllerAdvice**.

Q: How to handle exceptions globally in Spring Boot?

@RestControllerAdvice class with **@ExceptionHandler** methods that map exceptions to HTTP responses.

Q: Why use ResponseEntity?

Full control over the response: status code, headers, body. Useful when one endpoint can return multiple status codes.

Q: Status code for successful POST?

201 Created (with Location header pointing to the new resource).

Q: Status code for successful DELETE?

204 No Content typically, or 200 OK if returning something.

Q: When to return 404 vs 400?

404 — resource doesn't exist. 400 — request itself is malformed (bad JSON, missing fields, type errors).

Q: When to return 401 vs 403?

401 — caller is not authenticated (no/invalid credentials). 403 — authenticated but not authorized for this resource.

Q: Stateless API — what does it mean?

Server stores no client session state between requests. Each request must contain all info needed to process it.

Q: Should you expose entities in REST API?

No. Use DTOs. Reasons: hide internal fields, avoid coupling DB schema with API, prevent JPA lazy loading issues, control versioning.

Q: How does Spring Boot auto-configuration work?

On startup, Spring Boot scans classpath for @Conditional... beans defined in spring.factories / AutoConfiguration.imports. If conditions match (class present, property set) → auto-configures the relevant beans.

Q: How to disable a specific auto-configuration?

```
@SpringBootApplication(exclude = {DataSourceAutoConfiguration.class})
```

Q: How to change the default port?

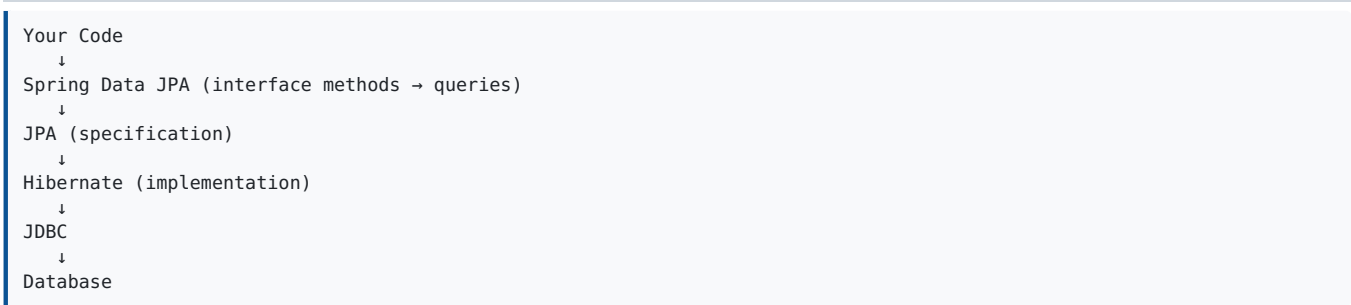
```
server.port=8090
```

 in application.properties. Or via env var: SERVER_PORT=8090.

Topic 4 — Spring Data JPA & Hibernate

4.1 JPA, Hibernate, Spring Data JPA — get them right

Layer	What it is
JPA (Java Persistence API)	A specification (interfaces + annotations). Just standards — no implementation.
Hibernate	An implementation of JPA. Most popular ORM framework.
Spring Data JPA	A Spring module that wraps JPA to eliminate repository boilerplate. Sits on top of Hibernate.



4.2 Entity Basics

```
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "user_name", nullable = false, length = 100)
    private String name;

    @Column(unique = true)
    private String email;

    @Enumerated(EnumType.STRING)
    private Status status;

    @Temporal(TemporalType.TIMESTAMP)
    private Date createdAt;

    @Transient
    private String tempField;           // not persisted

    @Lob
    private String description;        // CLOB
}
```

Required for JPA entity

- 1. @Entity annotation
- 2. No-arg constructor (public or protected)
- 3. Primary key (@Id)
- 4. Not final (Hibernate proxies subclass it)

@GeneratedValue strategies

Strategy	Description
IDENTITY	DB auto-increment column. Most common in MySQL.
SEQUENCE	Uses a DB sequence object (Postgres, Oracle). Most efficient.
TABLE	Stores ID in a separate table. Slowest, portable.
AUTO	Hibernate picks based on DB. Default but unpredictable.

4.3 Relationships (Critical!)

@OneToOne

```
@Entity
public class User {
    @OneToOne(cascade = CascadeType.ALL, orphanRemoval = true)
    @JoinColumn(name = "profile_id")
    private Profile profile;
}
```

@OneToMany and @ManyToOne (most common pair)

```
@Entity
public class User {
    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    private List<Order> orders;
}

@Entity
public class Order {
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id")
    private User user;
}
```

mappedBy = "the other side owns the relationship" — i.e., `Order.user` has the foreign key column.

@ManyToMany

```
@Entity
public class Student {
    @ManyToMany
    @JoinTable(name = "student_course",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id"))
    private List<Course> courses;
}

@Entity
public class Course {
    @ManyToMany(mappedBy = "courses")
    private List<Student> students;
}
```

Best practice: model @ManyToMany as two @OneToMany + @ManyToOne to a join entity if you ever need extra fields on the join.

4.4 Fetch Types — LAZY vs EAGER ⚠

Fetch Type	When	Default for
LAZY	Loaded on first access (proxy used)	@OneToMany, @ManyToMany
EAGER	Loaded immediately with parent	@ManyToOne, @OneToOne

Why defaults matter: Every time you fetch an Order, Hibernate auto-fetches the User too — extra query you didn't ask for. **Best practice:** always make @ManyToOne LAZY explicitly.

```
@ManyToOne(fetch = FetchType.LAZY) // ALWAYS make it lazy
private User user;
```

4.5 The N+1 Problem (MUST KNOW!)

The infamous code:

```
List<User> users = userRepo.findAll();
for (User u : users) {
    System.out.println(u.getOrders().size());
}
```

What's wrong?

This looks innocent but generates **catastrophic SQL traffic**.

Step-by-step what happens

Query 1 (the "+1"):

```
SELECT * FROM users;
```

Returns N users (say 100).

Then for EACH user, when you call `u.getOrders()` :

```
SELECT * FROM orders WHERE user_id = ?;    -- for user 1
SELECT * FROM orders WHERE user_id = ?;    -- for user 2
...
SELECT * FROM orders WHERE user_id = ?;    -- for user 100
```

Total: 1 + 100 = 101 queries to fetch what should be ONE query's worth of data.

Why "N+1"

- 1 query to load N parents
- N queries to load each parent's children
- Total = N + 1

Solutions

Solution 1: JOIN FETCH (JPQL)

```
@Query("SELECT u FROM User u LEFT JOIN FETCH u.orders")
List<User> findAllWithOrders();
```

Solution 2: @EntityGraph

```
@EntityGraph(attributePaths = "orders")
List<User> findAll();
```

Solution 3: @BatchSize

```
@OneToMany(mappedBy = "user")
@BatchSize(size = 50)
private List<Order> orders;
```

Solution 4: Use a projection / DTO (best if only counts needed)

```
@Query("SELECT new com.app.dto.UserOrderCount(u.id, u.name, COUNT(o)) " +
        "FROM User u LEFT JOIN u.orders o GROUP BY u.id, u.name")
List<UserOrderCount> findUserOrderCounts();
```

Interview phrasing

"This code triggers the N+1 query problem. `findAll()` fires one query for users. Since orders is lazy, accessing `getOrders()` on each user fires an additional query — so for N users we get N+1 queries. To fix: use JOIN FETCH in JPQL, or @EntityGraph, or restructure with a projection that returns just the count."

4.6 LazyInitializationException

```
@Service
public class UserService {
    @Transactional(readOnly = true)
    public User getUser(Long id) {
        return userRepo.findById(id).get();
    }
}

// Controller
@GetMapping("/users/{id}/orders")
public List<Order> getOrders(@PathVariable Long id) {
    User user = userService.getUser(id);
    return user.getOrders();    // ✗ LazyInitializationException!
}
```

Why?

- @Transactional opens a Hibernate session

- Session closes when `getUser()` returns
- `user.getOrders()` is a lazy proxy — needs an open session to query DB
- Session is gone → exception

Best practice

Use DTOs to expose data to controllers — never lazy-loaded entities.

4.7 Cascade Types

CascadeType	Effect
PERSIST	Save children when parent is saved
MERGE	Merge children when parent is merged
REMOVE	Delete children when parent is deleted
REFRESH	Refresh children when parent is refreshed
DETACH	Detach children when parent is detached
ALL	All of the above

orphanRemoval = true

If you remove an Order from `user.getOrders()` collection, the Order is **deleted from DB**.

4.8 JPA Lifecycle States

State	Meaning
Transient	New object, not in DB, no ID — <code>new User()</code>
Persistent / Managed	Tracked by EntityManager, in current session
Detached	Was managed, but session closed; still has ID
Removed	Marked for deletion in current session

save() vs persist() vs merge()

Method	Behavior
<code>persist(e)</code>	Insert new entity. Throws if entity exists.
<code>merge(e)</code>	Sync detached entity → returns new managed copy.
<code>save(e)</code> (Spring Data)	If transient → <code>persist</code> ; if detached → <code>merge</code> .

4.9 Spring Data JPA Repositories

Repository hierarchy

Repository
↓
CrudRepository (basic CRUD)
↓
PagingAndSortingRepository (pagination, sorting)
↓
JpaRepository (flush, batch, all JPA-specific)

Magic of Spring Data

```

public interface UserRepository extends JpaRepository<User, Long> {

    // Derived query – Spring parses method name into SQL!
    Optional<User> findByEmail(String email);
    List<User> findByDeptAndStatus(String dept, Status status);
    List<User> findByNameContainingIgnoreCase(String name);
    List<User> findByAgeGreaterThan(int age);
    long countByStatus(Status status);
    void deleteByEmail(String email);

    // JPQL query
    @Query("SELECT u FROM User u WHERE u.email = ?1")
    Optional<User> findByEmailManual(String email);

    // JPQL with named params
    @Query("SELECT u FROM User u WHERE u.dept = :dept")
    List<User> findInDepartment(@Param("dept") String dept);

    // Native SQL
    @Query(value = "SELECT * FROM users WHERE created_at > NOW() - INTERVAL 7 DAY",
            nativeQuery = true)
    List<User> findRecent();

    // Modifying query (for UPDATE/DELETE)
    @Modifying
    @Query("UPDATE User u SET u.status = :status WHERE u.id = :id")
    int updateStatus(@Param("id") Long id, @Param("status") Status status);
}

```

Derived query keywords

Keyword	Example	SQL
findBy	findByName(name)	WHERE name = ?
And / Or	findByNameAndAge(...)	WHERE name = ? AND age = ?
Containing / Like	findByNameContaining	WHERE name LIKE %?%
IgnoreCase	findByEmailIgnoreCase	LOWER(email) = LOWER(?)
GreaterThan / LessThan	findByAgeGreaterThan	WHERE age > ?
Between	findBySalaryBetween(a, b)	WHERE salary BETWEEN ? AND ?
OrderBy	findByDeptOrderByNameAsc	ORDER BY name ASC
Top / First	findTop3ByOrderBySalaryDesc	LIMIT 3
In / NotIn	findByIdIn(ids)	WHERE id IN (?)
IsNull	findByEmailIsNull	WHERE email IS NULL
True / False	findByActiveTrue	WHERE active = true

4.10 JPQL vs Native SQL vs Criteria API

JPQL (object-oriented)

```

@Query("SELECT u FROM User u WHERE u.dept = :dept")
List<User> findByDept(@Param("dept") String dept);

```

Native SQL

```

@Query(value = "SELECT * FROM users WHERE dept = ?1", nativeQuery = true)
List<User> findByDept(String dept);

```

Criteria API (programmatic, type-safe)

```

CriteriaBuilder cb = em.getCriteriaBuilder();
CriteriaQuery<User> cq = cb.createQuery(User.class);
Root<User> user = cq.from(User.class);
cq.select(user).where(cb.equal(user.get("dept"), "CSE"));
List<User> results = em.createQuery(cq).getResultList();

```

Used for **dynamic queries** built from variable conditions.

4.11 Hibernate Caches

First-level cache (always on)

- Per EntityManager / Session
- Same entity fetched twice in one transaction → second call returns cached object
- Cleared when transaction ends

Second-level cache (off by default, optional)

- Shared across sessions
- Configured via providers: EhCache, Caffeine, Hazelcast
- Useful for frequently read, rarely changed data

4.12 Common JPA Pitfalls (Interview Goldmines)

1. N+1 problem (covered)
2. LazyInitializationException (covered)
3. Bidirectional `toString()` causing infinite loop — exclude back-reference
4. `equals()` / `hashCode()` on entities — don't use auto-generated ID; use business key
5. Forgetting `@Transactional` → changes NOT saved (no flush)
6. `save()` returns a different object — always use the returned reference
7. Modifying detached entities silently fails
8. Cartesian product with multiple JOIN FETCH — use Set instead of List
9. Pagination + JOIN FETCH → in-memory pagination (Hibernate warns)

4.13 Database Schema Generation

```
spring.jpa.hibernate.ddl-auto=update    # update | create | create-drop | validate | none
```

Value	Behavior
none	Don't touch schema
validate	Compare schema, fail on mismatch (production safe)
update	Auto-add missing columns/tables (don't drop)
create	Drop and recreate on startup (test only)
create-drop	create on startup, drop on shutdown (testing)

Production: use Flyway or Liquibase for versioned migrations. Never `update` in production.

4.14 Auditing

```
@Entity
@EntityListeners(AuditingEntityListener.class)
public class User {
    @CreatedDate
    private LocalDateTime createdAt;
    @LastModifiedDate
    private LocalDateTime updatedAt;
    @CreatedBy
    private String createdBy;
    @LastModifiedBy
    private String modifiedBy;
}

@SpringBootApplication
@EnableJpaAuditing
public class Application { ... }
```

4.15 Top JPA / Hibernate Q&A

Q: Difference between JPA and Hibernate?

JPA is a specification (just interfaces and annotations). Hibernate is its most popular implementation. JPA is portable; Hibernate adds extra features.

Q: What is Spring Data JPA?

A Spring abstraction over JPA that eliminates repository boilerplate via interface-only repositories. Sits on top of JPA/Hibernate.

Q: Default fetch type for @OneToMany and @ManyToOne?

@OneToMany / @ManyToOne → LAZY. @ManyToOne / @OneToOne → EAGER.

Q: Why prefer LAZY by default?

Performance — don't load data you don't need. Avoid N+1 surprises and memory blowup.

Q: Explain N+1 problem and how to fix it.

Loading N parent entities then triggering N additional queries to fetch children (one per parent). Fix: JOIN FETCH, @EntityGraph, @BatchSize, projections.

Q: What is LazyInitializationException?

Trying to access a lazy-loaded collection after the Hibernate session has closed. Fix: use JOIN FETCH, @EntityGraph, or DTOs.

Q: save() vs persist() vs merge()?

persist: insert new (transient → managed); fails if ID exists. merge: detached → managed copy; returns the copy. save (Spring Data): persist if new, merge if detached.

Q: What is dirty checking?

Hibernate tracks managed entities. On flush/commit, it compares current state with snapshot from load. Changed fields → automatic UPDATE. No setter ≠ no save.

Q: First-level vs second-level cache?

L1: per-session, always on. L2: shared, optional, configurable provider.

Q: What is mappedBy?

Indicates the inverse (non-owning) side of a bidirectional relationship — the field name on the owning side that maps the FK.

Q: Cascade.ALL vs orphanRemoval?

CASCADE.ALL applies operations from parent to children. orphanRemoval deletes a child when removed from the parent's collection (even without parent removal).

Q: Difference between JPQL and SQL?

JPQL queries entity objects (not tables). Field names not column names. DB-portable. Hibernate translates to SQL.

Q: What is @Modifying?

Required for @Query annotations performing UPDATE/DELETE. Tells Spring Data this isn't a SELECT.

Q: How to implement pagination?

Use Pageable parameter and Page<T> return type. Spring binds ?page=&size=&sort=.

Q: What is @Version (optimistic locking)?

A version field auto-incremented on update. If version mismatches at commit → OptimisticLockException. Prevents lost updates without locking rows.

Q: Optimistic vs Pessimistic locking?

Optimistic: assume no conflicts, detect at commit time (@Version). Pessimistic: lock rows immediately (SELECT ... FOR UPDATE).

Q: What's the issue with equals/hashCode on JPA entities?

Auto-generated ID is null before persist → equals/hashCode behavior changes after save → entities lost in HashSets. Use a business key, or override carefully.

Q: What is "open-in-view"?

Spring Boot setting (default true) that keeps Hibernate session open during the entire HTTP request. Anti-pattern in production: encourages bad design, hides N+1 issues. Disable: `spring.jpa.open-in-view=false`.

Topic 5 — Spring Transactions

5.1 What is a Transaction?

A **transaction** is a sequence of operations on a database that must execute as a **single unit of work** — either ALL succeed (commit) or ALL fail (rollback).

Banking transfer example

1. Debit ₹1000 from Alice's account
2. Credit ₹1000 to Bob's account

If step 1 succeeds but step 2 fails, and we don't rollback step 1 — Alice loses ₹1000 and Bob never gets it. **Money vanished.**

5.2 ACID Properties

Property	Meaning
A — Atomicity	All operations succeed or none do (all-or-nothing)
C — Consistency	DB moves from one valid state to another (constraints, FK, business invariants preserved)
I — Isolation	Concurrent transactions don't interfere
D — Durability	Once committed, changes persist even if system crashes

5.3 @Transactional Annotation

Declarative way to manage transactions. Spring transparently:

1. Opens a transaction before method execution
2. Commits if method completes normally
3. Rolls back if method throws an unchecked exception
4. Closes the transaction after

```
@Service
public class TransferService {
    private final AccountRepository accountRepo;

    @Transactional
    public void transfer(Long fromId, Long toId, double amount) {
        Account from = accountRepo.findById(fromId).orElseThrow();
        Account to = accountRepo.findById(toId).orElseThrow();

        from.debit(amount);
        to.credit(amount);

        accountRepo.save(from);
        accountRepo.save(to);
        // No commit/rollback code – Spring handles it
    }
}
```

5.4 How @Transactional Works Under the Hood

1. Spring creates a **proxy** of your bean (CGLIB or JDK dynamic)
2. Proxy intercepts method calls
3. The proxy:
 - Opens a transaction (gets Connection, sets autoCommit=false)
 - Invokes your real method
 - On success → commits and returns
 - On unchecked exception → rolls back and rethrows

Caller → [Proxy: BEGIN TX] → Real Method → [Proxy: COMMIT or ROLLBACK]

⚠ **Self-invocation does NOT trigger transactions:**

```
public void method1() {
    method2();    // ✗ Direct call – bypasses proxy! No transaction!
}
@Transactional
public void method2() { /* DB ops */ }
```

Why? `method1()` calls `method2()` on `this` — which is the real object, not the proxy.

⚠ **@Transactional on private methods doesn't work.** Proxies can only intercept public methods. Private and final methods → annotation silently ignored.

5.5 Rollback Rules (Critical!)

Exception type	Rollback?
RuntimeException (unchecked)	✔ Yes (auto)
Error	✔ Yes (auto)
Checked exception	✗ NO (commits!)

This default catches developers off-guard. **Checked exceptions don't trigger rollback by default.**

Customizing rollback

```
@Transactional(rollbackFor = Exception.class)           // rollback on ANY exception
@Transactional(rollbackFor = {IOException.class, SQLException.class})
@Transactional(noRollbackFor = IllegalArgumentException.class)
```

Best practice: `@Transactional(rollbackFor = Exception.class)` for explicit, predictable behavior.

Trap example

```
@Transactional
public void process() throws IOException {
    accountRepo.save(account);    // saved
    throw new IOException("file missing");
    // ✗ Transaction COMMITS! Account is saved!
}
```

5.6 Propagation (Most Asked!)

Propagation = how a transactional method behaves when called within an EXISTING transaction.

Propagation	Behavior
REQUIRED (default)	Use existing TX if exists; else create new
REQUIRES_NEW	Always create NEW TX. Suspend caller's TX.
SUPPORTS	Use existing TX if exists; else run without TX
NOT_SUPPORTED	Suspend any existing TX; run without TX
MANDATORY	Use existing TX; throw if none exists
NEVER	Run without TX; throw if one exists
NESTED	Nested TX (savepoint) within existing TX

REQUIRES_NEW use case

```

@Transactional
public void placeOrder(Order order) {
    orderRepo.save(order);
    auditService.logAudit(order); // even if THIS fails, our save still committed
    throw new RuntimeException("oops"); // outer TX rolls back
}

@Transactional(propagation = Propagation.REQUIRES_NEW)
public void logAudit(Order order) {
    auditRepo.save(...);
    // committed independently, regardless of outer TX
}

```

5.7 Isolation Levels

Concurrency Anomalies

Anomaly	Description
Dirty Read	Reading uncommitted data from another TX
Non-Repeatable Read	Re-reading a row, getting different values
Phantom Read	Re-running a query, getting different rows
Lost Update	Two TXs update same row; one's update overwrites the other

4 isolation levels

Level	Dirty Read	Non-Repeatable	Phantom
READ_UNCOMMITTED	Possible	Possible	Possible
READ_COMMITTED	✗	Possible	Possible
REPEATABLE_READ	✗	✗	Possible
SERIALIZABLE	✗	✗	✗

Default by database

DB	Default
MySQL (InnoDB)	REPEATABLE_READ
PostgreSQL	READ_COMMITTED
Oracle	READ_COMMITTED
SQL Server	READ_COMMITTED

5.8 Read-Only Transactions

```

@Transactional(readOnly = true)
public List<User> findAll() { ... }

```

Benefits

- Performance — Hibernate skips dirty checking
- DB optimization — some DBs route to read replicas
- Safety — accidental writes blocked at runtime
- Documentation — communicates intent

Best practice: mark all read methods `readOnly = true`.

5.9 Common Pitfalls

1. @Transactional on private/final method → silently ignored
2. Self-invocation → no transaction
3. Checked exceptions don't rollback by default
4. Catching exceptions silently → no rollback
5. Long transactions (external API calls inside)

6. @Transactional on @PostConstruct doesn't work
7. Multiple data sources need explicit transactionManager
8. Read-only forgotten on read methods
9. Forgetting @EnableTransactionManagement (auto in Spring Boot)

5.10 Class-level vs Method-level

```
@Service
@Transactional(readOnly = true)           // safe default – read-only
public class UserService {

    public List<User> findAll() { ... }    // read-only inherited

    @Transactional                       // write methods override
    public User save(User u) { ... }
}
```

5.11 Top Spring Transaction Q&A

Q: What is a transaction?

A logical unit of work that groups multiple DB operations so they're either all committed or all rolled back. Enforces ACID properties.

Q: What are ACID properties?

Atomicity (all-or-nothing), Consistency (valid state to valid state), Isolation (concurrent TXs don't interfere), Durability (committed changes persist).

Q: What does @Transactional do?

Declaratively manages a transaction around a method. Spring proxies the bean; before the method, it begins a TX; on success it commits; on unchecked exception it rolls back; finally closes the TX.

Q: How does @Transactional work under the hood?

Spring creates a proxy (CGLIB or JDK dynamic). The proxy intercepts method calls and uses a PlatformTransactionManager to begin/commit/rollback transactions around the real method.

Q: Does @Transactional work on private methods?

No. Proxy can only intercept public method calls. Private/final/static methods are silently ignored.

Q: Does self-invocation trigger @Transactional?

No. `this.method()` calls the real object directly, bypassing the proxy. The annotation is ignored. Fix: extract to another bean.

Q: When does @Transactional NOT roll back?

By default, on checked exceptions. Only unchecked (RuntimeException, Error) trigger rollback automatically. Override with `rollbackFor = Exception.class`.

Q: What is Propagation?

Defines behavior when a transactional method is called within an existing transaction. 7 types: REQUIRED (default), REQUIRES_NEW, SUPPORTS, NOT_SUPPORTED, MANDATORY, NEVER, NESTED.

Q: Difference between REQUIRED and REQUIRES_NEW?

REQUIRED: join existing TX or create new. REQUIRES_NEW: always create new (suspends outer). Use REQUIRES_NEW for audit/log operations that must succeed independent of caller.

Q: What are isolation levels?

Define visibility of concurrent transactions' changes. Levels: READ_UNCOMMITTED (loosest), READ_COMMITTED, REPEATABLE_READ, SERIALIZABLE (strictest). Higher = safer but slower.

Q: Default isolation level?

Database-dependent. MySQL: REPEATABLE_READ. PostgreSQL/Oracle/SQL Server: READ_COMMITTED.

Q: What is dirty read?

TX A reads uncommitted changes from TX B. If B rolls back, A read invalid data. Prevented by READ_COMMITTED+.

Q: Why use readOnly = true?

Performance — Hibernate skips dirty checking. Some DBs route to replicas. Documents intent. Should mark all read methods.

Q: Where to put @Transactional — service or repository?

Service layer. Repositories often have multiple calls per service method that should be one logical TX. Service-level groups them.

Q: How to manually rollback?

```
TransactionAspectSupport.currentTransactionStatus().setRollbackOnly();
```

Q: How long should transactions be?

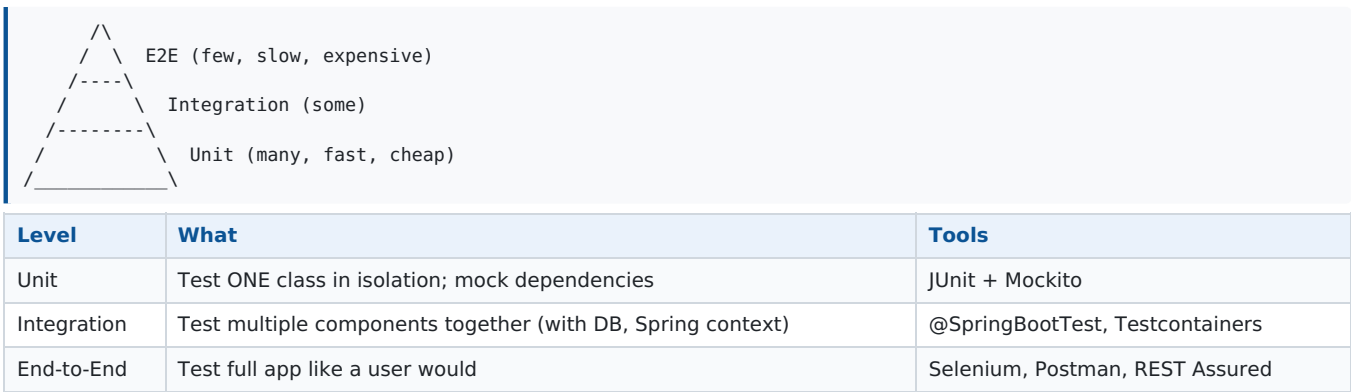
Short. Don't hold a TX during external API calls or long computations — locks held, connections wasted. Fetch data in TX, do external work outside.

Topic 6 — Testing: JUnit & Mockito

6.1 Why Test?

- 1. **Confidence to refactor** — change code knowing tests catch breakage
- 2. **Documentation** — tests show how code is supposed to be used
- 3. **Bug prevention** — catch issues before production
- 4. **Design feedback** — hard-to-test code = bad design
- 5. **Faster feedback loop** than manual testing

Testing Pyramid



6.2 JUnit 5 (Jupiter) Basics

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

Brings JUnit 5, Mockito, AssertJ, Hamcrest, JsonPath, Spring Test all in one.

Basic test

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class CalculatorTest {

    @Test
    void shouldAddTwoNumbers() {
        Calculator calc = new Calculator();
        int result = calc.add(2, 3);
        assertEquals(5, result);
    }

    @Test
    void shouldThrowOnDivideByZero() {
        Calculator calc = new Calculator();
        assertThrows(ArithmeticException.class, () -> calc.divide(10, 0));
    }
}
```

6.3 JUnit Annotations

Annotation	Purpose
@Test	Marks a test method
@BeforeEach	Runs before EACH test (setup)
@AfterEach	Runs after EACH test (cleanup)
@BeforeAll	Runs ONCE before all tests in class (must be static)
@AfterAll	Runs ONCE after all tests
@DisplayName("...")	Custom test name in reports
@Disabled	Skip a test
@Nested	Group related tests
@RepeatedTest(N)	Run test N times
@ParameterizedTest	Run test with multiple inputs
@Timeout(5)	Fail if test exceeds duration
@Tag("slow")	Filter tests by tag

Lifecycle example

```
class UserServiceTest {
    static Database db;
    UserService service;

    @BeforeAll
    static void setupAll() { db = new TestDatabase(); db.connect(); }

    @AfterAll
    static void teardownAll() { db.disconnect(); }

    @BeforeEach
    void setup() { service = new UserService(db); db.reset(); }

    @AfterEach
    void cleanup() { /* nothing usually */ }

    @Test
    void shouldRegisterUser() { /* ... */ }
    @Test
    void shouldFindUser() { /* ... */ }
}
```

6.4 Assertions

JUnit 5 native

```

assertEquals(expected, actual);
assertEquals(expected, actual, "Optional message");
assertEquals(0.1 + 0.2, 0.3, 0.0001);    // delta for doubles
assertNotEquals(...);
assertTrue(condition);
assertFalse(condition);
assertNull(value);
assertNotNull(value);
assertSame(obj1, obj2);                  // reference equality
assertNotSame(...);
assertArrayEquals(arr1, arr2);
assertIterableEquals(list1, list2);

// Multiple assertions
assertAll("user properties",
    () -> assertEquals("Alice", user.getName()),
    () -> assertEquals(25, user.getAge()),
    () -> assertTrue(user.isActive())
);

// Exception assertions
ArithmeticException ex = assertThrows(ArithmeticException.class,
    () -> calc.divide(1, 0));
assertEquals("/ by zero", ex.getMessage());

// Timeout
assertTimeout(Duration.ofMillis(100), () -> service.fastOperation());
assertTimeoutPreemptively(Duration.ofSeconds(1), () -> service.op());

```

AssertJ (much more readable, recommended)

```

import static org.assertj.core.api.Assertions.*;

assertThat(user.getName()).isEqualTo("Alice");
assertThat(user.getAge()).isGreaterThan(18).isLessThan(65);
assertThat(users).hasSize(3).contains(alice).doesNotContain(bob);
assertThat(map).containsEntry("key", "value");
assertThat(name).isNotNull().isNotEmpty().startsWith("A");
assertThat(date).isAfter(yesterday).isBefore(tomorrow);

// Fluent exception assertions
assertThatThrownBy(() -> service.divide(1, 0))
    .isInstanceOf(ArithmeticException.class)
    .hasMessageContaining("zero");

```

6.5 Parameterized Tests

```

@ParameterizedTest
@ValueSource(ints = {1, 2, 3, 4, 5})
void shouldBePositive(int n) {
    assertThat(n).isPositive();
}

@ParameterizedTest
@ValueSource(strings = {"alice", "bob", "carol"})
void shouldBeNonEmpty(String name) {
    assertThat(name).isNotEmpty();
}

@ParameterizedTest
@CsvSource({
    "1, 1, 2",
    "2, 3, 5",
    "10, -5, 5"
})
void shouldAddCorrectly(int a, int b, int expected) {
    assertThat(calc.add(a, b)).isEqualTo(expected);
}

@ParameterizedTest
@MethodSource("provideTestData")
void test(int input, int expected) { /* ... */ }

static Stream<Arguments> provideTestData() {
    return Stream.of(Arguments.of(1, 1), Arguments.of(2, 4));
}

```

6.6 Mockito Basics

Mockito creates fake (mock) objects to simulate dependencies, letting you test ONE class in isolation.

```
@Service
public class UserService {
    private final UserRepository userRepository;
    private final EmailService emailService;
    public UserService(UserRepository u, EmailService e) {
        this.userRepository = u; this.emailService = e;
    }

    public User register(String name) {
        if (userRepository.existsByName(name))
            throw new DuplicateUserException(name);
        User user = userRepository.save(new User(name));
        emailService.sendWelcome(user);
        return user;
    }
}
```

Mockito test

```
@ExtendWith(MockitoExtension.class)
class UserServiceTest {

    @Mock UserRepository userRepository;
    @Mock EmailService emailService;
    @InjectMocks UserService userService; // creates and injects mocks

    @Test
    void shouldRegisterNewUser() {
        // Given
        when(userRepository.existsByName("Alice")).thenReturn(false);
        User savedUser = new User(1L, "Alice");
        when(userRepository.save(any(User.class))).thenReturn(savedUser);

        // When
        User result = userService.register("Alice");

        // Then
        assertThat(result.getName()).isEqualTo("Alice");
        verify(userRepository).save(any(User.class));
        verify(emailService).sendWelcome(savedUser);
    }

    @Test
    void shouldThrowOnDuplicate() {
        when(userRepository.existsByName("Bob")).thenReturn(true);
        assertThatThrownBy(() -> userService.register("Bob"))
            .isInstanceOf(DuplicateUserException.class);
        verify(emailService, never()).sendWelcome(any());
    }
}
```

6.7 Mockito Annotations

Annotation	Purpose
@Mock	Create a mock
@Spy	Real object, can override methods
@InjectMocks	Auto-inject mocks/spies into the target
@Captor	Capture argument for inspection
@ExtendWith(MockitoExtension.class)	Enable Mockito with JUnit 5

6.8 Stubbing Methods

```
// Return a value
when(repo.findById(1L)).thenReturn(Optional.of(user));

// Multiple returns (consecutive calls)
when(repo.count()).thenReturn(0L, 1L, 2L);

// Throw exception
when(repo.findById(99L)).thenThrow(new EntityNotFoundException());

// For void methods
doNothing().when(emailService).send(any());
doThrow(new RuntimeException()).when(emailService).send(any());

// Argument matchers
when(repo.findByName(anyString())).thenReturn(user);
when(repo.find(eq(1L), anyString())).thenReturn(user); // mix specific and matchers
```

Argument matcher rule: if you use ANY matcher in a stubbing, ALL arguments must use matchers. Use `eq()` for specific values.

6.9 Verifying Interactions

```
verify(repo).save(user); // called exactly once
verify(repo, times(2)).save(any()); // exactly 2 times
verify(repo, never()).delete(any()); // never called
verify(repo, atLeastOnce()).find(anyLong());
verify(repo, atMost(3)).find(anyLong());

// Order verification
InOrder order = inOrder(repo, emailService);
order.verify(repo).save(user);
order.verify(emailService).sendWelcome(user);

// Verify no unexpected interactions
verifyNoMoreInteractions(repo);
verifyNoInteractions(emailService);
```

6.10 Argument Captors

```
@Captor ArgumentCaptor<User> userCaptor;

@Test
void shouldSaveCorrectUser() {
    userService.register("Alice");

    verify(userRepository).save(userCaptor.capture());
    User saved = userCaptor.getValue();
    assertThat(saved.getName()).isEqualTo("Alice");
    assertThat(saved.isActive()).isTrue();
}
```

6.11 @Mock vs @Spy

@Mock	@Spy
Fully fake — methods return defaults (null, 0, false) unless stubbed	Real object — calls real methods unless stubbed
Replaces all behavior	Wraps a real object
Most common	Use sparingly — for partial mocks

```
@Spy List<String> spyList = new ArrayList<>();
spyList.add("hello");
verify(spyList).add("hello");
when(spyList.size()).thenReturn(100);
assertThat(spyList.size()).isEqualTo(100);
assertThat(spyList.get(0)).isEqualTo("hello"); // real behavior
```

6.12 Testing Spring Boot Apps

@SpringBootTest — full integration test

```
@SpringBootTest
class UserServiceIT {
    @Autowired UserService userService;
    @Autowired UserRepository userRepository;

    @Test
    @Transactional // rollback after test
    void shouldRegister() {
        User user = userService.register("Alice");
        assertThat(userRepository.count()).isEqualTo(1);
    }
}
```

Loads entire Spring context → slower but realistic.

@DataJpaTest — repositories only

```
@DataJpaTest
class UserRepositoryTest {
    @Autowired TestEntityManager em;
    @Autowired UserRepository repo;

    @Test
    void shouldFindByEmail() {
        User user = new User("Alice", "alice@x.com");
        em.persistAndFlush(user);
        assertThat(repo.findByEmail("alice@x.com")).isPresent();
    }
}
```

Only loads JPA components. Uses in-memory H2 by default. Each test rolls back.

@WebMvcTest — controller layer only

```
@WebMvcTest(UserController.class)
class UserControllerTest {

    @Autowired MockMvc mockMvc;
    @MockBean UserService userService; // mocks the service in Spring context

    @Test
    void shouldGetUser() throws Exception {
        when(userService.findById(1L)).thenReturn(new User(1L, "Alice"));

        mockMvc.perform(get("/users/1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.name").value("Alice"));
    }

    @Test
    void shouldCreateUser() throws Exception {
        mockMvc.perform(post("/users")
            .contentType(MediaType.APPLICATION_JSON)
            .content("{\"name\":\"Alice\"}"))
            .andExpect(status().isCreated());
    }
}
```

6.13 @MockBean vs @Mock

@Mock	@MockBean
Mockito-only	Spring + Mockito
Plain unit tests	Spring tests (@SpringBootTest, @WebMvcTest)
Initialized by MockitoExtension	Replaces real bean in Spring ApplicationContext

6.14 Mocking Static Methods (Mockito 3.4+)

```
// Need: mockito-inline dependency
@Test
void shouldMockStatic() {
    try (MockedStatic<LocalDate> mock = mockStatic(LocalDate.class)) {
        mock.when(LocalDate::now).thenReturn(LocalDate.of(2024, 1, 1));
        assertThat(LocalDate.now()).isEqualTo(LocalDate.of(2024, 1, 1));
    }
}
```

6.15 Test Best Practices

1. **One assertion per concept** (multiple physical assertions OK)
2. **Arrange-Act-Assert** (Given-When-Then) structure
3. **Descriptive names** — `shouldThrowWhenUserIsNull()` not `test1()`
4. **Independence** — tests don't depend on order
5. **Fast** — unit tests should run in milliseconds
6. **Deterministic** — never flaky
7. **Test behavior, not implementation** — don't verify private method calls
8. **Don't mock what you don't own** — use thin wrappers around third-party APIs and mock the wrapper
9. **Use builders or factories** for test data
10. **Test edge cases** — null, empty, boundaries

6.16 Test Data Builders

```
public class UserTestBuilder {
    private String name = "DefaultName";
    private int age = 25;
    private boolean active = true;

    public UserTestBuilder name(String name) { this.name = name; return this; }
    public UserTestBuilder age(int age) { this.age = age; return this; }
    public UserTestBuilder inactive() { this.active = false; return this; }

    public User build() { return new User(name, age, active); }

    public static UserTestBuilder aUser() { return new UserTestBuilder(); }
}

// Use:
User user = aUser().name("Alice").age(30).inactive().build();
```

6.17 Code Coverage

JaCoCo measures % of code covered by tests.

```
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.11</version>
</plugin>
```

Run: `mvn test jacoco:report` → `target/site/jacoco/index.html`

Coverage rule of thumb: 70-80% line coverage is healthy. 100% is rarely worth it. Focus on critical paths.

6.18 Top Testing Q&A

Q: Difference between unit and integration tests?

Unit: tests one class in isolation, mocks deps, fast (ms). Integration: tests multiple components together, often loads Spring context or real DB, slower.

Q: Why use Mockito?

Replace dependencies with controlled fakes — isolate the unit being tested, control return values, verify interactions, simulate failures.

Q: @Mock vs @InjectMocks?

@Mock creates a mock of a class. @InjectMocks creates the actual class under test and injects mocks into it (via constructor, setter, or field).

Q: @Mock vs @Spy?

@Mock is fully fake. @Spy wraps a real object — calls real methods unless stubbed.

Q: @Mock vs @MockBean?

@Mock is for Mockito (plain unit tests). @MockBean is Spring — replaces a bean in the application context.

Q: Difference between @BeforeEach and @BeforeAll?

@BeforeEach runs before EVERY test. @BeforeAll runs ONCE before all tests in the class (must be static).

Q: How to test exceptions in JUnit 5?

`assertThrows(SomeException.class, () -> method())`. Or AssertJ: `assertThatThrownBy(...).isInstanceOf(...)`.

Q: How to verify a method was called?

`verify(mock).method(args)`. Add `times(n)`, `never()`, `atLeast()` for counts.

Q: How to mock void methods?

`doNothing().when(mock).method()` or `doThrow(...).when(mock).method()`.

Q: How to capture arguments passed to a mock?

@Captor ArgumentCaptor + `verify(mock).method(captor.capture())`, then `captor.getValue()`.

Q: Why @SpringBootTest is slow?

Loads entire Spring context. Use sliced tests (@DataJpaTest, @WebMvcTest) for layer-specific tests; reserve @SpringBootTest for true E2E integration.

Q: What is MockMvc?

Spring's testing tool for controllers — sends fake HTTP requests through the controller layer without starting a real server.

Q: Difference between @WebMvcTest and @SpringBootTest?

@WebMvcTest only loads the web layer (controllers + filters); @SpringBootTest loads everything. WebMvcTest is much faster.

Q: What is Testcontainers?

Library that runs real DBs (PostgreSQL, MySQL, etc.) in Docker containers for tests. More realistic than H2 in-memory.

Q: Code coverage — what's a good %?

70-80% line coverage is reasonable. 100% is rarely worth chasing. Focus on critical business logic, not getters/setters.

Q: What's a flaky test?

A test that sometimes passes, sometimes fails without code change. Causes: timing, shared state, randomness, external dependencies. Fix or delete — they erode confidence.

Q: Should you mock everything?

No. Only mock dependencies that are slow, unreliable, or have side effects. Mocking pure value objects or trivial classes is anti-pattern.

Q: TDD — what is it?

Test-Driven Development: write failing test → write minimal code to pass → refactor. "Red, Green, Refactor" cycle.

Q: How to test private methods?

Don't directly. Test through the public method that uses them. If a private method is complex enough to test alone, extract it to a separate class with public API.

Q: How to mock final classes/methods?

Use mockito-inline dependency. Standard Mockito can't mock final by default.

Topic 7 — Design Patterns

7.1 What are Design Patterns?

Design patterns are typical, reusable solutions to common problems in software design. Catalogued by the "Gang of Four" (GoF) in 1994. Three categories:

Category	Purpose	Examples
Creational	Object creation	Singleton, Factory, Builder, Prototype
Structural	Composition of classes/objects	Adapter, Decorator, Facade, Proxy
Behavioral	Communication between objects	Strategy, Observer, Template, Command, Iterator

7.2 Singleton Pattern (Creational)

Intent: Ensure a class has only ONE instance and provide a global point of access.

1. Eager Initialization

```
public class Singleton {
    private static final Singleton INSTANCE = new Singleton();
    private Singleton() {}
    public static Singleton getInstance() { return INSTANCE; }
}
```

Simple, thread-safe. Created even if never used (wasteful for heavy objects).

2. Lazy (NOT thread-safe!)

```
public class Singleton {
    private static Singleton instance;
    private Singleton() {}
    public static Singleton getInstance() {
        if (instance == null) instance = new Singleton();
        return instance;
    }
}
```

3. Synchronized (thread-safe but slow)

```
public static synchronized Singleton getInstance() {
    if (instance == null) instance = new Singleton();
    return instance;
}
```

4. Double-Checked Locking (DCL) — recommended for lazy

```
public class Singleton {
    private static volatile Singleton instance;
    private Singleton() {}
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

volatile is critical — prevents reordering issues.

5. Bill Pugh — best for most cases

```
public class Singleton {
    private Singleton() {}
    private static class Holder {
        private static final Singleton INSTANCE = new Singleton();
    }
    public static Singleton getInstance() { return Holder.INSTANCE; }
}
```

Lazy + thread-safe + no synchronization overhead. Uses JVM's class loading.

6. Enum — the safest

```
public enum Singleton {  
    INSTANCE;  
    public void doSomething() { /* ... */ }  
}
```

Joshua Bloch's recommendation. Handles serialization & reflection attacks for free.

Real-world uses

- Spring beans (singleton scope)
- Logger
- Configuration manager
- Connection pool
- Cache

7.3 Factory Method Pattern (Creational)

Intent: Create objects without exposing the instantiation logic.

```
public interface Shape { void draw(); }  
class Circle implements Shape { public void draw() { System.out.println("Circle"); } }  
class Square implements Shape { public void draw() { System.out.println("Square"); } }  
  
public class ShapeFactory {  
    public static Shape create(String type) {  
        return switch (type) {  
            case "CIRCLE" -> new Circle();  
            case "SQUARE" -> new Square();  
            default -> throw new IllegalArgumentException("Unknown: " + type);  
        };  
    }  
}  
  
// Usage  
Shape s = ShapeFactory.create("CIRCLE");  
s.draw();
```

Where used in JDK: `Calendar.getInstance()`, `NumberFormat.getInstance()`, `DriverManager.getConnection()`.

7.4 Abstract Factory Pattern (Creational)

Intent: Factory of factories — produces families of related objects.

```
interface Button { void render(); }  
interface Checkbox { void render(); }  
  
class WindowsButton implements Button { public void render() { /*...*/ } }  
class MacButton implements Button { public void render() { /*...*/ } }  
  
interface UIFactory {  
    Button createButton();  
    Checkbox createCheckbox();  
}  
  
class WindowsFactory implements UIFactory {  
    public Button createButton() { return new WindowsButton(); }  
    public Checkbox createCheckbox() { return new WindowsCheckbox(); }  
}  
  
class MacFactory implements UIFactory {  
    public Button createButton() { return new MacButton(); }  
    public Checkbox createCheckbox() { return new MacCheckbox(); }  
}
```

7.5 Builder Pattern (Creational)

Intent: Construct complex objects step by step. Especially when many optional parameters.

```

public class User {
    private final String name;
    private final int age;
    private final String email;
    private final String phone;
    private final String address;

    private User(Builder b) {
        this.name = b.name; this.age = b.age;
        this.email = b.email; this.phone = b.phone; this.address = b.address;
    }

    public static class Builder {
        private String name; private int age;
        private String email; private String phone; private String address;
        public Builder name(String n) { this.name = n; return this; }
        public Builder age(int a) { this.age = a; return this; }
        public Builder email(String e) { this.email = e; return this; }
        public Builder phone(String p) { this.phone = p; return this; }
        public Builder address(String a) { this.address = a; return this; }
        public User build() { return new User(this); }
    }
}

// Usage
User u = new User.Builder()
    .name("Alice").age(30).email("a@x.com").build();

```

Why Builder? Telescoping constructors are unmaintainable. Lombok's `@Builder` generates this automatically.

Where in JDK: `StringBuilder`, `StringBuffer`.

7.6 Prototype Pattern (Creational)

Intent: Create new objects by cloning existing ones.

```

class Document implements Cloneable {
    String title; List<String> pages;

    @Override
    public Document clone() {
        try {
            Document copy = (Document) super.clone();
            copy.pages = new ArrayList<>(this.pages); // deep copy
            return copy;
        } catch (CloneNotSupportedException e) { throw new AssertionError(); }
    }
}

```

7.7 Adapter Pattern (Structural)

Intent: Make incompatible interfaces work together.

```

// Existing class with incompatible interface
class LegacyPrinter {
    void printText(String text) { System.out.println(text); }
}

// New interface our app expects
interface ModernPrinter { void print(String content); }

// Adapter
class LegacyPrinterAdapter implements ModernPrinter {
    private final LegacyPrinter legacy;
    public LegacyPrinterAdapter(LegacyPrinter l) { this.legacy = l; }
    public void print(String content) { legacy.printText(content); }
}

```

Where in JDK: `Arrays.asList()`, `InputStreamReader` (adapts `InputStream` → `Reader`).

7.8 Decorator Pattern (Structural)

Intent: Add behavior to objects dynamically without modifying their class.

```

interface Coffee {
    double cost();
    String description();
}

class SimpleCoffee implements Coffee {
    public double cost() { return 5.0; }
    public String description() { return "Coffee"; }
}

abstract class CoffeeDecorator implements Coffee {
    protected Coffee coffee;
    public CoffeeDecorator(Coffee c) { this.coffee = c; }
}

class MilkDecorator extends CoffeeDecorator {
    public MilkDecorator(Coffee c) { super(c); }
    public double cost() { return coffee.cost() + 1.0; }
    public String description() { return coffee.description() + ", Milk"; }
}

class SugarDecorator extends CoffeeDecorator {
    public SugarDecorator(Coffee c) { super(c); }
    public double cost() { return coffee.cost() + 0.5; }
    public String description() { return coffee.description() + ", Sugar"; }
}

// Usage
Coffee coffee = new SugarDecorator(new MilkDecorator(new SimpleCoffee()));
// "Coffee, Milk, Sugar" - $6.50

```

Where in JDK: `BufferedReader(new FileReader(...))`, `Collections.unmodifiableList()`.

7.9 Facade Pattern (Structural)

Intent: Provide a simplified interface to a complex subsystem.

```

// Complex subsystems
class CPU { void execute() { /*...*/ } }
class Memory { void load() { /*...*/ } }
class HardDrive { void read() { /*...*/ } }

// Facade
class Computer {
    private CPU cpu = new CPU();
    private Memory memory = new Memory();
    private HardDrive hd = new HardDrive();

    public void start() {
        hd.read();
        memory.load();
        cpu.execute();
    }
}

// Client just calls computer.start() instead of orchestrating all 3

```

7.10 Proxy Pattern (Structural)

Intent: Stand-in object that controls access to another object.

Types:

- **Virtual Proxy** — lazy initialization
- **Protection Proxy** — access control
- **Remote Proxy** — represents remote object (RMI)
- **Smart Proxy** — additional behavior (logging, caching)

```

interface Image { void display(); }

class RealImage implements Image {
    private String filename;
    public RealImage(String f) { this.filename = f; loadFromDisk(); }
    private void loadFromDisk() { System.out.println("Loading " + filename); }
    public void display() { System.out.println("Displaying " + filename); }
}

class ImageProxy implements Image {
    private RealImage realImage;
    private String filename;
    public ImageProxy(String f) { this.filename = f; }
    public void display() {
        if (realImage == null) realImage = new RealImage(filename);    // lazy load
        realImage.display();
    }
}

```

Where used: Spring AOP, Hibernate lazy loading, RMI.

7.11 Strategy Pattern (Behavioral)

Intent: Define a family of algorithms, encapsulate each, make them interchangeable.

```

interface PaymentStrategy {
    void pay(double amount);
}

class CreditCardStrategy implements PaymentStrategy {
    public void pay(double a) { System.out.println("Paid " + a + " via credit card"); }
}

class PaypalStrategy implements PaymentStrategy {
    public void pay(double a) { System.out.println("Paid " + a + " via PayPal"); }
}

class ShoppingCart {
    private PaymentStrategy strategy;
    public void setStrategy(PaymentStrategy s) { this.strategy = s; }
    public void checkout(double amount) { strategy.pay(amount); }
}

// Usage
ShoppingCart cart = new ShoppingCart();
cart.setStrategy(new CreditCardStrategy());
cart.checkout(100);

cart.setStrategy(new PaypalStrategy());
cart.checkout(50);

```

Where in Java: `Comparator` passed to `Collections.sort()`.

7.12 Observer Pattern (Behavioral)

Intent: One-to-many dependency. When subject's state changes, all observers are notified automatically.

```

interface Observer { void update(String news); }

class NewsAgency {
    private List<Observer> observers = new ArrayList<>();
    private String latestNews;

    public void register(Observer o) { observers.add(o); }
    public void unregister(Observer o) { observers.remove(o); }

    public void publish(String news) {
        this.latestNews = news;
        observers.forEach(o -> o.update(news));
    }
}

class NewsChannel implements Observer {
    private String name;
    public NewsChannel(String n) { this.name = n; }
    public void update(String news) {
        System.out.println(name + " received: " + news);
    }
}

```

Where used: Spring's ApplicationEventPublisher, Java Swing listeners, RxJava.

7.13 Template Method Pattern (Behavioral)

Intent: Define the skeleton of an algorithm in a base class; let subclasses override specific steps.

```

abstract class DataProcessor {
    // Template method (final – can't be overridden)
    public final void process() {
        readData();
        validateData();
        transformData();
        saveData();
    }

    protected abstract void readData();
    protected abstract void transformData();

    // Common implementation
    protected void validateData() { System.out.println("Validating..."); }
    protected void saveData() { System.out.println("Saving..."); }
}

class CsvProcessor extends DataProcessor {
    protected void readData() { System.out.println("Reading CSV"); }
    protected void transformData() { System.out.println("Transforming CSV"); }
}

```

Where used: Spring's JdbcTemplate, RestTemplate, JpaRepository.

7.14 Command Pattern (Behavioral)

Intent: Encapsulate a request as an object — supports queueing, logging, undo/redo.

```

interface Command {
    void execute();
}

class TurnOnLight implements Command {
    private Light light;
    public TurnOnLight(Light l) { this.light = l; }
    public void execute() { light.turnOn(); }
}

class RemoteControl {
    private Command command;
    public void setCommand(Command c) { this.command = c; }
    public void pressButton() { command.execute(); }
}

```

7.15 Iterator Pattern (Behavioral)

Intent: Sequentially access elements of a collection without exposing its internal structure.

Built into Java: `java.util.Iterator`, used by every Collection. `for-each` loop uses it under the hood.

7.16 Chain of Responsibility (Behavioral)

Intent: Pass a request along a chain of handlers — each decides to handle or pass along.

```
abstract class Handler {
    protected Handler next;
    public Handler setNext(Handler n) { this.next = n; return n; }
    public abstract void handle(Request request);
}

class AuthHandler extends Handler {
    public void handle(Request r) {
        if (!r.isAuthenticated()) throw new AuthException();
        if (next != null) next.handle(r);
    }
}

class RateLimitHandler extends Handler {
    public void handle(Request r) {
        if (r.tooManyAttempts()) throw new RateLimitException();
        if (next != null) next.handle(r);
    }
}
```

Where used: Servlet filters, Spring Security filter chain, exception handling.

7.17 Top Design Patterns Q&A

Q: What is a design pattern?

A reusable solution to a recurring problem in software design. Three categories: creational, structural, behavioral.

Q: Why use design patterns?

Common vocabulary, proven solutions, cleaner architecture, easier maintenance, better extensibility.

Q: Best Singleton implementation?

Bill Pugh (Holder class) for general use — lazy + thread-safe without sync overhead. Enum is even safer (handles serialization & reflection).

Q: Why is volatile needed in DCL Singleton?

Without volatile, JIT/CPU may reorder instructions: another thread could see a non-null but partially constructed object. volatile provides the happens-before guarantee.

Q: How can Singleton be broken?

Reflection (private constructor accessible), serialization (deserialization creates new instance), cloning, multiple ClassLoaders. Enum prevents all of these.

Q: Difference between Factory and Abstract Factory?

Factory creates ONE type of product. Abstract Factory is a "factory of factories" — creates families of related products.

Q: When to use Builder?

Object has many fields, especially optional ones, where telescoping constructors become unwieldy. Use Lombok's @Builder for free.

Q: Adapter vs Decorator?

Adapter changes the INTERFACE of an existing class. Decorator adds new BEHAVIOR while keeping the interface same.

Q: Decorator vs Inheritance?

Decorator is dynamic — combine behaviors at runtime. Inheritance is static — fixed at compile time. Decorator avoids "class explosion" with many feature combinations.

Q: Where do you use Strategy in real code?

Anywhere you have algorithm variants — payment processors, sorting comparators, validation rules, discount calculators. Spring DI naturally enables this.

Q: How does Spring use design patterns?

Heavily: Singleton (default bean scope), Factory (BeanFactory), Proxy (AOP, @Transactional), Template Method (JdbcTemplate), Observer (ApplicationEvent), Strategy (DI choosing impl), Decorator (BeanPostProcessor).

Q: Where does Hibernate use Proxy?

Lazy loading. When you fetch an entity with a lazy association, Hibernate returns a proxy. Accessing a method on it triggers a DB query.

Q: Builder vs Constructor?

Constructor: simple cases, all fields known at compile time. Builder: many optional fields, complex construction, fluent API; immutable result.

Q: Pattern used by JDBC DriverManager?

Factory (creates Connection objects).

Q: Pattern used by Comparator?

Strategy.

Q: Pattern used by InputStreamReader wrapping InputStream?

Adapter (Reader interface $\leftrightarrow$ InputStream byte source).

Q: Pattern used by BufferedReader wrapping FileReader?

Decorator — adds buffering behavior.

Q: How are MVC and Observer related?

In classic MVC, Views observe Model and update when it changes. The Observer pattern is at the heart.

Q: What's an anti-pattern?

A common but counterproductive solution. Examples: Singleton overuse (global state), God Class, Spaghetti Code.

Q: Common pitfalls of Singleton?

Hidden global state; hard to test (can't easily mock); concurrency issues; tightly couples client to a specific class. Often DI is a better solution.

Topic 8 — SOLID Principles

8.1 What is SOLID?

Five principles of object-oriented design coined by Robert C. Martin (Uncle Bob). They make code **maintainable, flexible, testable, and scalable**.

Letter	Principle
S	Single Responsibility
O	Open/Closed
L	Liskov Substitution
I	Interface Segregation
D	Dependency Inversion

8.2 S — Single Responsibility Principle (SRP)

"A class should have only ONE reason to change."

A class should do **one thing** and do it well. Multiple responsibilities = multiple reasons to change = fragile code.

✗ Violation

```
class Employee {  
    void calculateSalary() { /* business logic */ }  
    void saveToDatabase() { /* persistence */ }  
    void generateReport() { /* presentation */ }  
    void sendEmail() { /* notification */ }  
}
```

4 reasons to change. Hard to test, modify, or scale.

✓ Fix — separate responsibilities

```
class Employee { /* domain object */ }  
class SalaryCalculator { double calculate(Employee e) { /* ... */ } }  
class EmployeeRepository { void save(Employee e) { /* ... */ } }  
class ReportGenerator { void generate(Employee e) { /* ... */ } }  
class EmailService { void sendEmail(Employee e, String msg) { /* ... */ } }
```

Benefits: easier testing, focused changes, reusable, parallelizable team work.

8.3 O — Open/Closed Principle (OCP)

"Software entities should be OPEN for extension, but CLOSED for modification."

You should be able to add new functionality without touching existing, working code.

✗ Violation

```
class AreaCalculator {  
    public double calculate(Object shape) {  
        if (shape instanceof Rectangle r) return r.width * r.height;  
        if (shape instanceof Circle c) return Math.PI * c.radius * c.radius;  
        // Adding Triangle? Modify this class!  
        return 0;  
    }  
}
```

✓ Fix — use polymorphism

```

interface Shape { double area(); }

class Rectangle implements Shape {
    double width, height;
    public double area() { return width * height; }
}
class Circle implements Shape {
    double radius;
    public double area() { return Math.PI * radius * radius; }
}
class Triangle implements Shape { // NEW shape – no modification needed
    double base, height;
    public double area() { return 0.5 * base * height; }
}

class AreaCalculator {
    public double calculate(Shape s) { return s.area(); }
}

```

8.4 L — Liskov Substitution Principle (LSP)

"Subtypes must be substitutable for their base types without breaking the program."

If `Square` extends `Rectangle`, anywhere code uses `Rectangle`, you should be able to pass a `Square` without surprises.

✗ Classic violation — Square inherits from Rectangle

```

class Rectangle {
    protected int width, height;
    public void setWidth(int w) { this.width = w; }
    public void setHeight(int h) { this.height = h; }
    public int area() { return width * height; }
}

class Square extends Rectangle {
    @Override
    public void setWidth(int w) { this.width = w; this.height = w; } // breaks invariant
    @Override
    public void setHeight(int h) { this.width = h; this.height = h; }
}

// Caller expects Rectangle behavior:
void test(Rectangle r) {
    r.setWidth(5);
    r.setHeight(10);
    assert r.area() == 50; // FAILS for Square (returns 100!)
}

```

✓ Fix — don't force inheritance where it doesn't fit

`Square` is NOT a `Rectangle` in OO terms (mutable). Use a common `Shape` interface or composition instead.

8.5 I — Interface Segregation Principle (ISP)

"Clients should not be forced to depend on methods they don't use."

Prefer many small, focused interfaces over one fat interface.

✗ Violation — fat interface

```

interface Worker {
    void work();
    void eat();
    void sleep();
}

class Robot implements Worker {
    public void work() { /* fine */ }
    public void eat() { throw new UnsupportedOperationException(); } // ✗
    public void sleep() { throw new UnsupportedOperationException(); } // ✗
}

```

✓ Fix — split into focused interfaces

```
interface Workable { void work(); }
interface Eatable { void eat(); }
interface Sleepable { void sleep(); }

class Human implements Workable, Eatable, Sleepable { /* all 3 */ }
class Robot implements Workable { public void work() { /* ... */ } }
```

8.6 D — Dependency Inversion Principle (DIP)

"Depend on abstractions, not concretions. High-level modules should not depend on low-level modules — both should depend on abstractions."

✗ Violation

```
class MySQLDatabase {
    public void save(String data) { /* MySQL specific */ }
}

class UserService {
    private MySQLDatabase db = new MySQLDatabase(); // ✗ tightly coupled
    public void register(User u) { db.save(u.toString()); }
}
```

Switching to PostgreSQL? Rewrite UserService. Testing? Can't mock easily.

✓ Fix — abstraction (interface)

```
interface Database {
    void save(String data);
}

class MySQLDatabase implements Database { public void save(String d) { /* ... */ } }
class PostgresDatabase implements Database { public void save(String d) { /* ... */ } }

class UserService {
    private final Database db;
    public UserService(Database db) { this.db = db; } // injected
    public void register(User u) { db.save(u.toString()); }
}
```

Now: easy to swap implementations, easy to test (inject mock), high-level (UserService) doesn't know about low-level (MySQL).

Spring's DI is fundamentally an implementation of DIP.

8.7 Comprehensive Example — All 5 Principles Together

Building a Notification System:

```
// SRP: Each class has one job

// ISP: Small focused interfaces
interface NotificationChannel {
    void send(String to, String message);
}

interface MessageTemplate {
    String build(String userName, String content);
}

// OCP: Open for extension via implementations, closed for modification
class EmailChannel implements NotificationChannel {
    public void send(String to, String msg) { /* SMTP */ }
}

class SmsChannel implements NotificationChannel {
    public void send(String to, String msg) { /* Twilio */ }
}

class SlackChannel implements NotificationChannel { // new – no existing code modified
    public void send(String to, String msg) { /* Slack API */ }
}

// LSP: All channels are substitutable for NotificationChannel
class WelcomeMessageTemplate implements MessageTemplate {
    public String build(String name, String content) {
        return "Welcome " + name + "! " + content;
    }
}

// DIP: NotificationService depends on abstractions, not concrete classes
class NotificationService {
    private final NotificationChannel channel;
    private final MessageTemplate template;

    public NotificationService(NotificationChannel ch, MessageTemplate t) {
        this.channel = ch; this.template = t;
    }

    public void notify(String to, String name, String content) {
        String message = template.build(name, content);
        channel.send(to, message);
    }
}

// Use any channel + any template combination
NotificationService email = new NotificationService(
    new EmailChannel(), new WelcomeMessageTemplate());

NotificationService sms = new NotificationService(
    new SmsChannel(), new WelcomeMessageTemplate());
```

8.8 Other Useful Principles (Bonus)

DRY — Don't Repeat Yourself

Every piece of knowledge should have a single, unambiguous representation.

KISS — Keep It Simple, Stupid

Simplest solution that works. Avoid over-engineering.

YAGNI — You Aren't Gonna Need It

Don't add functionality until necessary. Premature flexibility = waste.

Composition over Inheritance

Prefer "has-a" relationships over "is-a". Inheritance is rigid; composition is flexible.

Law of Demeter

"Talk only to your immediate friends." Avoid `obj.getX().getY().getZ().doSomething()` chains — they couple you to a deep object structure.

8.9 Top SOLID Q&A

Q: What does SOLID stand for?

Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion. 5 OO design principles by Robert C. Martin.

Q: Why are SOLID principles important?

Make code maintainable, testable, flexible, reusable. Reduce coupling, increase cohesion, ease change.

Q: Single Responsibility Principle?

A class should have only ONE reason to change — one cohesive responsibility. Multiple responsibilities lead to fragile, hard-to-maintain code.

Q: Open/Closed Principle?

Software should be open for extension, closed for modification. Add new behavior via new classes (subclasses, implementations) without changing existing code.

Q: How do you achieve Open/Closed?

Through abstraction (interfaces, abstract classes) and polymorphism. Code against interfaces; new requirements = new implementations.

Q: Liskov Substitution Principle?

Subtypes must be substitutable for their base types without breaking program correctness. Subclasses can't violate the parent's contract.

Q: Classic LSP violation example?

Square extending Rectangle and overriding setters to keep both dimensions equal — breaks Rectangle's invariants. Fix: use composition or a shared Shape interface.

Q: Interface Segregation Principle?

Clients shouldn't depend on methods they don't use. Many small, focused interfaces are better than one fat interface.

Q: Dependency Inversion Principle?

Depend on abstractions, not on concretions. High-level modules should not depend on low-level modules; both should depend on abstractions (interfaces).

Q: How does Spring help with DIP?

DI container injects concrete implementations into classes that depend only on interfaces. Spring is essentially DIP institutionalized.

Q: Difference between DI and DIP?

DIP is the principle (depend on abstractions). DI is one technique to achieve it (inject the dependency). You can follow DIP without DI (factory pattern, service locator).

Q: SRP vs ISP — aren't they the same?

No. SRP is about CLASSES having one responsibility. ISP is about INTERFACES being focused (clients don't depend on irrelevant methods). Different scopes.

Q: Can SOLID be over-applied?

Yes. Excessive abstraction creates indirection that hurts readability. Apply SOLID where flexibility matters; for simple, stable code, simpler is better.

Q: Composition over Inheritance — what does it mean?

Prefer "has-a" relationships over "is-a". Inheritance creates tight coupling; composition is flexible. E.g., Car HAS-A Engine instead of Car EXTENDS Vehicle.

Q: DRY — what is it?

Don't Repeat Yourself. Every piece of knowledge should have a single, unambiguous representation. Duplication = maintenance burden + bugs from inconsistency.

Q: YAGNI?

You Aren't Gonna Need It. Don't add functionality until you actually need it. Premature flexibility is waste.

Q: Real-world example of OCP?

A payment processor with PaymentStrategy interface. Adding a new payment type (Bitcoin) only requires a new BitcoinStrategy class — no existing code changes.

Q: Why is high coupling bad?

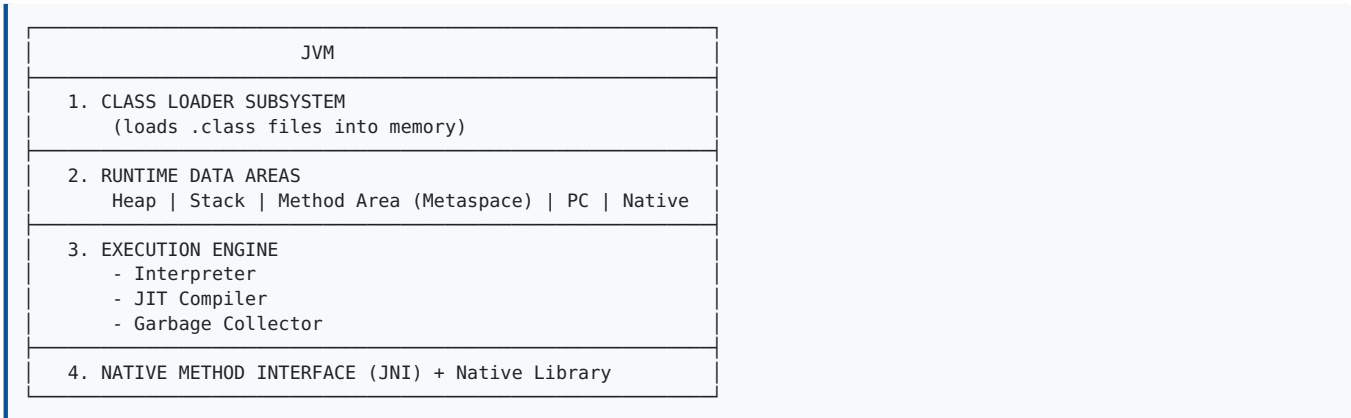
Changes ripple. Hard to test (can't mock). Hard to reuse. Hard to understand. SOLID minimizes coupling.

Q: How do you apply SOLID in Spring Boot apps?

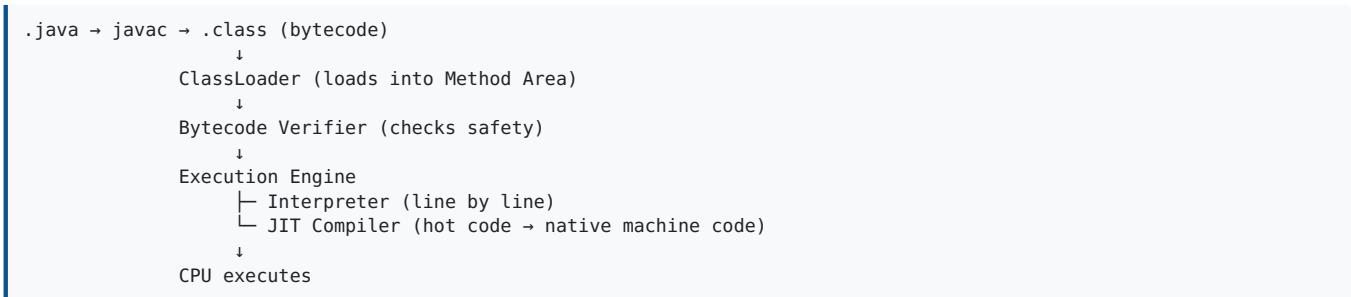
SRP — Service, Repository, Controller separation. OCP — Strategy implementations as Spring beans. LSP — interfaces with multiple impls. ISP — separate Repositories. DIP — constructor injection of interfaces.

Topic 9 — JVM Internals

9.1 JVM Architecture (Big Picture)



How a Java program runs (high level)



9.2 ClassLoader Subsystem

A component of the JVM responsible for **loading .class files into memory** when needed (lazy loading).

Three phases of class loading

Phase 1: Loading

- Find .class file
- Read bytes
- Create a Class<T> object in Method Area (Metaspace)

Phase 2: Linking (3 sub-steps)

- **Verify** — bytecode safety checks (e.g., no stack overflows, valid types)
- **Prepare** — allocate memory for static fields, set to default values (0, null, false)
- **Resolve** — convert symbolic references to actual references

Phase 3: Initialization

- Execute static initializers and `static {}` blocks
- Assign actual values to static fields

```
class Demo {
    static int x = 10;           // prepared as 0, then initialized to 10
    static int y;
    static { y = 20; }
}
```

9.3 ClassLoader Hierarchy

Bootstrap ClassLoader	(loads core JDK – rt.jar / java.base module)
↑	
Platform ClassLoader	(Java 9+) (loads java.* modules)
↑	
System / Application	(loads classes from CLASSPATH)
↑	
Custom ClassLoader	(user-defined, e.g., for plugin systems)

1. Bootstrap ClassLoader

- The first ClassLoader, written in C/C++ (part of native JVM)
- Loads core JDK classes: `java.lang.*`, `java.util.*`, etc.
- `String.class.getClassLoader()` returns null because Bootstrap is native

2. Platform ClassLoader (Java 9+) / Extension ClassLoader (Java 8)

- Loads platform-specific (extension) classes
- Java 8: from `JAVA_HOME/jre/lib/ext`
- Java 9+: from `java.base` and other platform modules

3. Application/System ClassLoader

- Loads classes from CLASSPATH (your application classes, third-party libraries)
- Default for application code

4. Custom ClassLoader

- User-defined for plugin systems, hot reloading (Tomcat web apps), bytecode manipulation

9.4 Parent Delegation Model

When asked to load a class, a ClassLoader first delegates to its parent. Only if the parent can't find it does the child try.

```
Request: load com.myapp.Foo
  ↓ delegates to parent
System ClassLoader → Platform ClassLoader → Bootstrap ClassLoader
                                     ↓ "I don't have it"
Bootstrap returns null
Platform tries... fails
Platform returns null
System loads from CLASSPATH ✓
```

Why delegation matters — security

Imagine someone defines `java.lang.String` in their JAR. Without delegation, that malicious class might shadow the real one. With delegation, Bootstrap **always loads java.lang.String first** → user-defined version ignored.

Other principles

- **Visibility:** Child ClassLoader can see classes loaded by its parent, but NOT vice versa.
- **Uniqueness:** A class loaded by one ClassLoader is distinct from the same class loaded by a different ClassLoader. Even if names match.

```
Class<?> c1 = loaderA.loadClass("com.app.Foo");
Class<?> c2 = loaderB.loadClass("com.app.Foo");
c1 == c2;           // false! different classes from JVM's perspective
```

This is how Tomcat isolates web apps — each app has its own ClassLoader → can have different versions of same library.

9.5 Custom ClassLoader (advanced)

```

public class FileSystemClassLoader extends ClassLoader {
    private final String classPath;

    public FileSystemClassLoader(String path) { this.classPath = path; }

    @Override
    protected Class<?> findClass(String name) throws ClassNotFoundException {
        try {
            byte[] bytes = Files.readAllBytes(
                Paths.get(classPath, name.replace('.', '/') + ".class"));
            return defineClass(name, bytes, 0, bytes.length);
        } catch (IOException e) {
            throw new ClassNotFoundException(name, e);
        }
    }
}

```

Real use cases

- Plugin systems (IntelliJ, Eclipse)
- Hot reload (Spring Boot DevTools)
- App servers isolating web apps (Tomcat)
- OSGi bundles
- Bytecode manipulation (Hibernate proxies, AspectJ load-time weaving)

9.6 Bytecode (the .class file)

Bytecode = intermediate, platform-neutral instructions that the JVM executes. Each instruction is 1-5 bytes.

View bytecode with javap

```

javac Hello.java
javap -c Hello      # disassemble bytecode
javap -p -v Hello   # verbose, including private members

```

Example

```

public class Hello {
    public static void main(String[] args) {
        int sum = 2 + 3;
        System.out.println(sum);
    }
}

```

```

$ javap -c Hello
public static void main(java.lang.String[]);
  Code:
    0: iconst_2           // push 2 onto stack
    1: iconst_3           // push 3
    2: iadd              // pop 2, pop 3, push 5
    3: istore_1           // store stack top to local var 1 (sum)
    4: getstatic          #2      // System.out
    7: iload_1            // load sum
    8: invokevirtual      #3      // println(int)
   11: return

```

Common bytecode instructions

Category	Instructions
Stack operations	iconst_*, iload_*, istore_*
Arithmetic	iadd, isub, imul, idiv
Method invocation	invokevirtual, invokestatic, invokespecial, invokeinterface, invokedynamic
Control flow	goto, if_icmpeq, tableswitch
Object creation	new, dup, getfield, putfield

invokedynamic (Java 7+): the gateway to lambdas. JVM defers method binding to runtime, enabling efficient lambda creation without generating new classes.

9.7 Execution Engine — Interpreter vs JIT

Interpreter

Reads and executes bytecode one instruction at a time.

- ✓ Fast startup
- ✗ Slow execution

JIT (Just-In-Time) Compiler

Detects frequently executed code ("hotspots") and compiles them to native machine code at runtime, cached for reuse.

- ✓ Near-native performance after warm-up
- ✗ Slow startup (compilation cost)

Tiered compilation (HotSpot)

Tier	What
0	Interpreter
1	C1 compiler (fast, light optimization)
2	C1 with limited profiling
3	C1 with full profiling
4	C2 compiler (slow, heavy optimization)

JIT optimizations (the magic)

Optimization	What it does
Inlining	Replace method call with method body — eliminates call overhead
Escape analysis	If object never escapes method → allocate on stack (no heap, no GC)
Lock elision	Removes synchronization on objects that don't escape
Lock coarsening	Merges adjacent synchronized blocks into one
Dead code elimination	Removes unreachable code
Loop unrolling	for(i=0;i<4;i++) x++; → x++;x++;x++;x++; (no loop overhead)
Branch prediction	Optimizes likely branches
Constant folding	int x = 2 + 3; becomes int x = 5; at compile time

C1 vs C2

- **C1 (Client compiler)** — fast compile, less optimization, used for short-lived apps
- **C2 (Server compiler)** — slower compile, aggressive optimization, used for long-running servers
- Tiered compilation uses BOTH: C1 first for quick wins, then C2 for hotspots

GraalVM / AOT

Ahead-of-time (AOT) compilation — compile to native code BEFORE running. Sub-second startup. Used by Spring Native, Quarkus.

9.8 Java Memory Model (JMM)

JMM defines how threads interact through memory — when one thread's writes become visible to another, and what reorderings are allowed.

Key concepts

1. Visibility

Without proper synchronization, one thread's write may never be seen by another thread (CPU caches, register optimizations).

```
boolean running = true;
new Thread() -> { while (running) {} }).start(); // may loop forever!
running = false; // change may not be visible to other thread

// Fix:
volatile boolean running = true;
```

2. Atomicity

Some operations are NOT atomic:

- `count++` (read + add + write — 3 steps)
- `long` / `double` writes on 32-bit JVMs

3. Reordering

JIT/CPU may reorder instructions for performance. Multi-threaded → reordering can break invariants.

```
int x = 0; boolean ready = false;

// Thread A
x = 42;
ready = true;

// Thread B
while (!ready) {}
System.out.println(x); // could print 0! (reordering)
```

Happens-Before Relationship — the heart of JMM

Rule	Action A happens-before Action B
Program order	Within a single thread, statements happen-before later statements
Monitor lock	Unlock happens-before subsequent lock on same monitor
Volatile	Write to volatile happens-before subsequent read of same volatile
Thread start	Thread.start() happens-before any action in started thread
Thread join	All actions in thread happen-before another's join() returns
Final fields	Final field write in constructor happens-before another thread reading the reference (after safe publication)
Transitivity	If A→B and B→C, then A→C

Practical implications

- volatile — provides visibility (and prevents reordering)
- synchronized — provides visibility + mutual exclusion
- final — ensures correct construction publication
- Atomic classes — built on CAS, provide happens-before

9.9 HotSpot JVM Optimizations (deeper)

Escape Analysis

Determines if an object's lifetime is confined to its method.

```
public int sum(int n) {
    Point p = new Point(0, 0); // doesn't escape
    p.x = 1; p.y = n;
    return p.x + p.y;
}
```

JIT may eliminate the heap allocation entirely — store p.x, p.y in registers/stack.

Scalar Replacement

After escape analysis, the JVM may "break apart" non-escaping objects into their fields, allocating them as scalars on stack.

No heap allocation, no GC pressure.

Inlining (most impactful)

Method calls are expensive (stack frame setup). JIT replaces calls with method bodies for small/hot methods.

```
public int square(int x) { return x * x; }

// Before inlining: int result = square(5);
// After inlining:  int result = 5 * 5;    // = 25 (then constant-folded)
```

Profile-Guided Optimization (PGO)

JVM collects runtime profiles (which branches are hot, which types are common at virtual call sites) and uses them to specialize compilation. Static AOT compilers can't match this for long-running servers.

9.10 GC Internals

- **Young Gen** → Eden + Survivors (S0, S1)
- **Old Gen** → tenured long-lived objects
- **Metaspace** → class metadata (Java 8+, replaces PermGen)
- Minor GC (Young) frequent, fast
- Major/Full GC (Old + Metaspace) infrequent, slow

GC algorithms: Serial, Parallel, CMS (deprecated), G1 (default Java 9+), ZGC, Shenandoah.

9.11 JVM Tuning Flags (production essentials)

Heap sizing

```
-Xms512m           # initial heap
-Xmx2g             # max heap
-XX:NewRatio=2     # Old:Young = 2:1
-XX:SurvivorRatio=8 # Eden:S0:S1 = 8:1:1
-Xss512k          # thread stack size
```

Choose GC

```
-XX:+UseG1GC           # G1 (default Java 9+)
-XX:+UseParallelGC     # Parallel
-XX:+UseZGC             # Ultra-low pause
-XX:+UseShenandoahGC   #
-XX:MaxGCPauseMillis=200 # G1 pause goal
```

Diagnostics

```
-XX:+HeapDumpOnOutOfMemoryError # dump on OOM
-XX:HeapDumpPath=/tmp/heap.hprof
-XX:+PrintGCDetails             # GC logs (Java 8)
-Xlog:gc*:file=gc.log          # GC logs (Java 9+)
-XX:+PrintCompilation           # JIT activity
-XX:+UnlockDiagnosticVMOptions
-XX:+PrintInlining              # see JIT inlining
```

Common tuning advice

- Set `-Xms = -Xmx` in production (avoid resize jitter)
- Don't set heap > 32GB without compressed oops consideration
- Profile first; don't tune blindly

9.12 Tools to Inspect a Running JVM

Tool	Purpose
jps	List Java processes (PIDs)
jstat	Live GC statistics
jmap	Heap dump, histogram
jstack	Thread dump
jcmd	All-in-one (recommended modern tool)
jconsole	GUI JMX monitoring
VisualVM	GUI profiler
Java Flight Recorder (JFR)	Built-in production profiling, low overhead
Mission Control (JMC)	Analyze JFR recordings
Eclipse MAT	Heap dump analyzer
async-profiler	Sampling profiler (very accurate)

Examples

```
jps -l                                # list Java processes
jstat -gc <pid> 1000                  # GC stats every 1s
jmap -dump:live,format=b,file=heap.hprof <pid>
jstack <pid>                           # thread dump
jcmd <pid> GC.heap_dump heap.hprof
jcmd <pid> JFR.start duration=60s filename=app.jfr
```

9.13 Reflection & MethodHandle

Reflection

Inspect and modify program structure at runtime: classes, fields, methods, constructors.

```
Class<?> clazz = Class.forName("com.app.User");
Constructor<?> ctor = clazz.getDeclaredConstructor();
ctor.setAccessible(true);           // bypass private!
Object instance = ctor.newInstance();

Field field = clazz.getDeclaredField("name");
field.setAccessible(true);
field.set(instance, "Alice");

Method method = clazz.getMethod("toString");
String s = (String) method.invoke(instance);
```

Use cases

- Frameworks (Spring, Hibernate, JUnit) — need to introspect user classes
- Serialization (Jackson)
- Plugin systems

Drawbacks

- Performance — slower than direct calls
- Security — bypasses access control
- Type safety — runtime errors instead of compile-time

Modern alternative — MethodHandle (Java 7+)

```
MethodHandles.Lookup lookup = MethodHandles.lookup();
MethodHandle handle = lookup.findVirtual(String.class, "toUpperCase",
    MethodType.methodType(String.class));
String result = (String) handle.invoke("hello");
```

Faster than reflection. Used by invokedynamic, lambdas internally.

9.14 Annotations Processing

Three retention policies

Policy	When
SOURCE	Discarded by compiler (e.g., @Override)
CLASS	In .class file, not at runtime (default)
RUNTIME	Available via reflection at runtime (e.g., Spring's @Service)

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface MyAnnotation {
    String value() default "";
}
```

Spring scans classpath at startup, uses reflection to find @Component/@Service/@Autowired etc., wires beans accordingly.

9.15 Java Modules (JPMS, Java 9+)

```

module com.myapp {
    requires java.sql;
    requires spring.core;

    exports com.myapp.api;
    exports com.myapp.model to com.myapp.tests;
}

```

Benefits

- Strong encapsulation
- Smaller JREs via jlink
- Faster startup
- Reliable configuration

Reality: Most Spring/enterprise apps still use the classpath (not modules) due to migration cost.

9.16 Top JVM Internals Q&A

Q: What is the JVM?

Java Virtual Machine — runtime that executes Java bytecode. Has ClassLoader, Memory areas, Execution Engine (interpreter + JIT + GC), and JNI.

Q: How does Java achieve platform independence?

Java compiles to bytecode (platform-neutral). JVM translates bytecode to platform-specific machine code. Each OS has its own JVM build.

Q: What is bytecode?

Intermediate, platform-neutral instructions stored in .class files. Each instruction 1-5 bytes. Stack-based — operates on an operand stack and local variables.

Q: Phases of class loading?

Loading (find/read .class), Linking (verify, prepare, resolve), Initialization (static init).

Q: ClassLoader hierarchy?

Bootstrap (native, core JDK) → Platform/Extension (JDK platform classes) → System/Application (CLASSPATH) → Custom.

Q: Why does `String.class.getClassLoader()` return null?

String is loaded by Bootstrap ClassLoader, written in native (C/C++) and not exposed as a Java object → returns null by convention.

Q: What is parent delegation?

When a ClassLoader is asked to load a class, it first delegates to its parent. Only loads itself if parent fails. Provides security (prevents user classes from shadowing core JDK) and uniqueness.

Q: Can two ClassLoaders load the same class?

Yes — they produce different Class objects, treated as distinct types by JVM. This is how Tomcat isolates web apps with different library versions.

Q: What is JIT compilation?

Just-In-Time — JVM detects "hot" code and compiles it to native machine code at runtime, cached for reuse. Combines startup speed of interpreter with peak performance of native code.

Q: Difference between C1 and C2 compilers?

C1 (Client) — fast compile, light optimization. C2 (Server) — slower, aggressive optimization. Tiered compilation uses both: C1 for warm-up, C2 for steady-state hotspots.

Q: What is escape analysis?

JIT optimization that determines if an object's lifetime is confined to a method. If yes, JIT may allocate on stack (or scalar-replace) → no heap allocation, no GC pressure.

Q: What is method inlining?

JIT replaces a method call with the actual method body, eliminating call overhead. Most impactful optimization for small/hot methods.

Q: What is the Java Memory Model (JMM)?

Specification defining how threads interact through memory — visibility, ordering, atomicity guarantees. Defines "happens-before" rules.

Q: What is happens-before?

A relation between memory operations that guarantees ordering and visibility. If A happens-before B, A's effects are visible to B. Established by synchronized, volatile, final, Thread.start/join, atomics.

Q: Why do we need volatile?

Without it, threads may cache values in registers/CPU caches → updates from other threads invisible. volatile ensures reads always go to main memory and writes are flushed → cross-thread visibility.

Q: Difference between volatile and synchronized?

volatile = visibility only, no atomicity. synchronized = visibility + atomicity + mutual exclusion + blocking.

Q: Where is bytecode stored at runtime?

In the Method Area (called Metaspace since Java 8). Method Area is part of non-heap memory.

Q: What is Metaspace?

Replaces PermGen (Java 8+). Stores class metadata in native memory (outside heap). Auto-grows by default.

Q: How does the JVM handle method dispatch?

Bytecode instructions: invokestatic (static), invokespecial (constructors, private, super), invokevirtual (instance methods, dynamic dispatch), invokeinterface (interface methods), invokedynamic (runtime-bound, lambdas).

Q: How are lambdas implemented under the hood?

Via invokedynamic — JVM bootstraps a method handle for the lambda body lazily. Avoids generating a new class file per lambda. More efficient than anonymous classes.

Q: Reflection vs MethodHandle?

Reflection: API for runtime introspection, slower, dynamic. MethodHandle: faster, type-safe, used internally by lambda + invokedynamic.

Q: What's the difference between PermGen and Metaspace?

PermGen — fixed-size part of the heap, prone to OOM on classloader leaks. Metaspace (Java 8+) — native memory, auto-grows, more flexible.

Q: What are GC roots?

Local variables in active methods, static fields, active threads, JNI references, synchronization monitors held by threads.

Q: Tiered compilation explained?

Code starts in interpreter (Tier 0), gets profiled, climbs to C1 (Tiers 1-3), then C2 (Tier 4) if hot enough. Combines fast startup with peak performance.

Q: How does synchronized work internally?

Each object has a monitor (mark word in object header). synchronized uses monitorenter and monitorexit bytecode instructions. JVM optimizes via biased locking, lightweight locking, lock elision.

Q: What does GraalVM offer?

A polyglot VM that supports AOT (Ahead-of-Time) compilation → produces native executables with sub-second startup. Used by Spring Native, Quarkus.

Q: Difference between JIT and AOT?

JIT — compiles at runtime, adapts to runtime profile (better long-run performance). AOT — compiles before runtime, fast startup, smaller memory footprint (good for serverless, CLI).

Q: Why is a Java application slow on first call (cold start)?

ClassLoading on demand (lazy); Interpreter mode initially; JIT hasn't compiled hot code yet; HotSpot needs to gather profile data.

Q: Java vs Kotlin/Scala on JVM — same bytecode?

Yes — all JVM languages compile to JVM bytecode. They share the same runtime, GC, ClassLoader, JIT. Differences are at the language and library level.

9 topics · Spring · JPA · Testing · Patterns · SOLID · JVM — you've got the depth.